GPU Architectures 2

Agenda

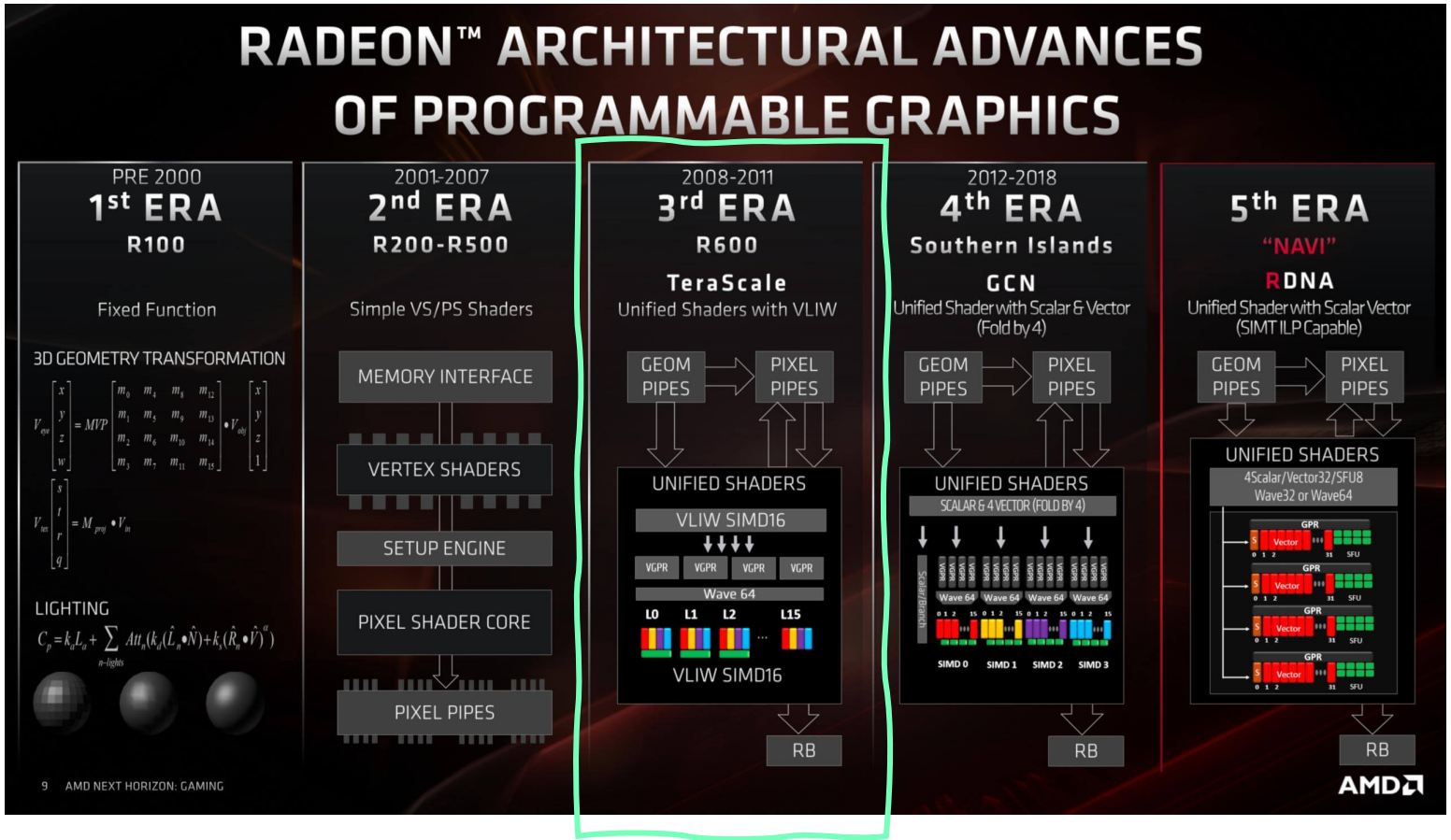
- Understand the GPU execution model
  - SIMT (vs. SIMD)

- Learn the details of GPU architectures
  - NVIDIA GPU architecture
  - AMD GPU architecture
  - ARM GPU architecture

AMD GPU Architectures

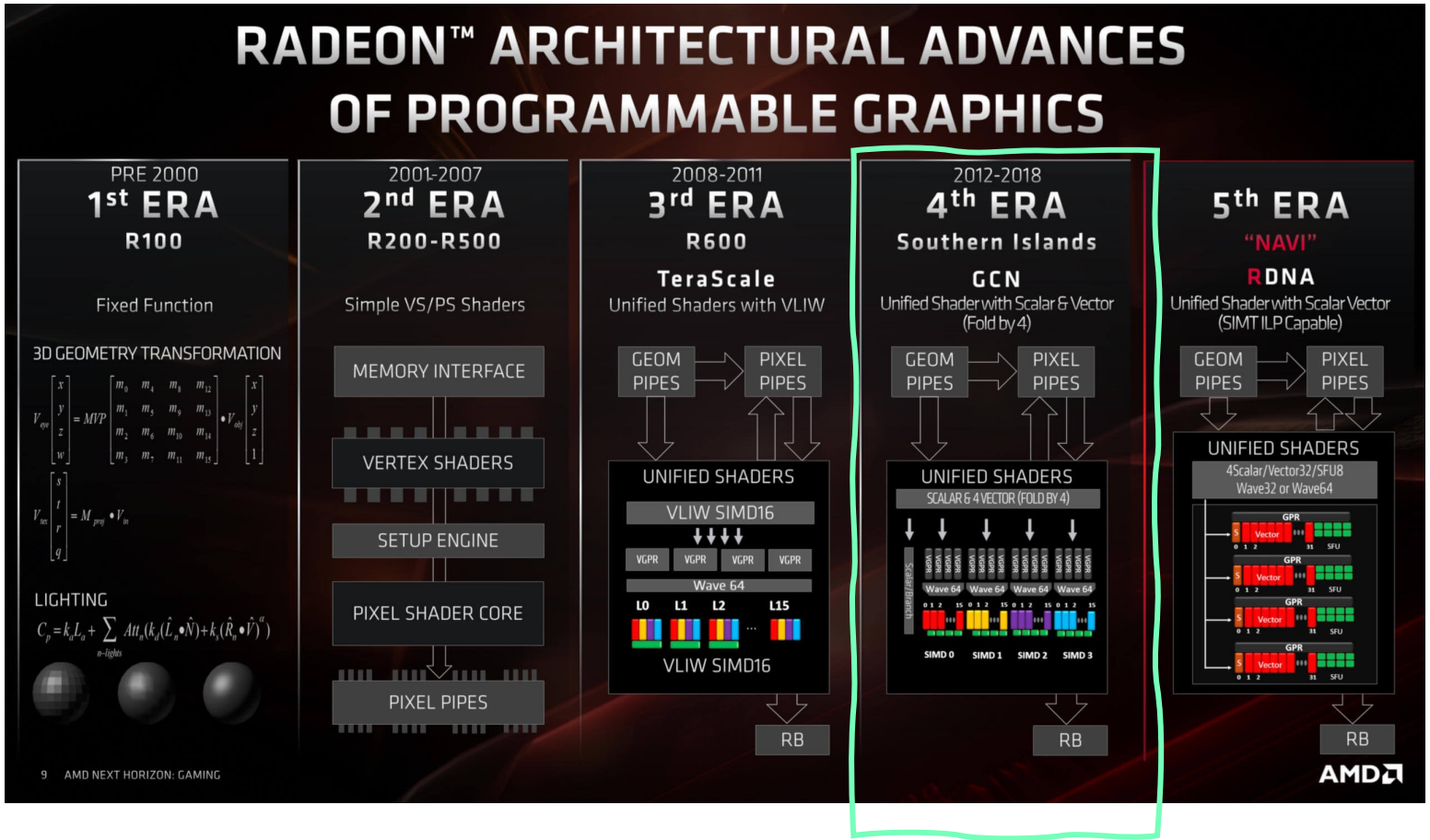
Arctehictural Evolution

- TeraScale (2007)
  - After acquiring ATI (vendor of Radeon GPU), the first AMD GPU architecture with general-purpose computing support
  - Supports OpenCL , OpenGL, DirectX
  - VLIW (Very Long Instruction Word) SIMD architecture
    - Compiler statically packs independent instructions into an instruction
    - Hardware unpacks and executes them in parallel
    - Complicates compiler design and wastes compute resources



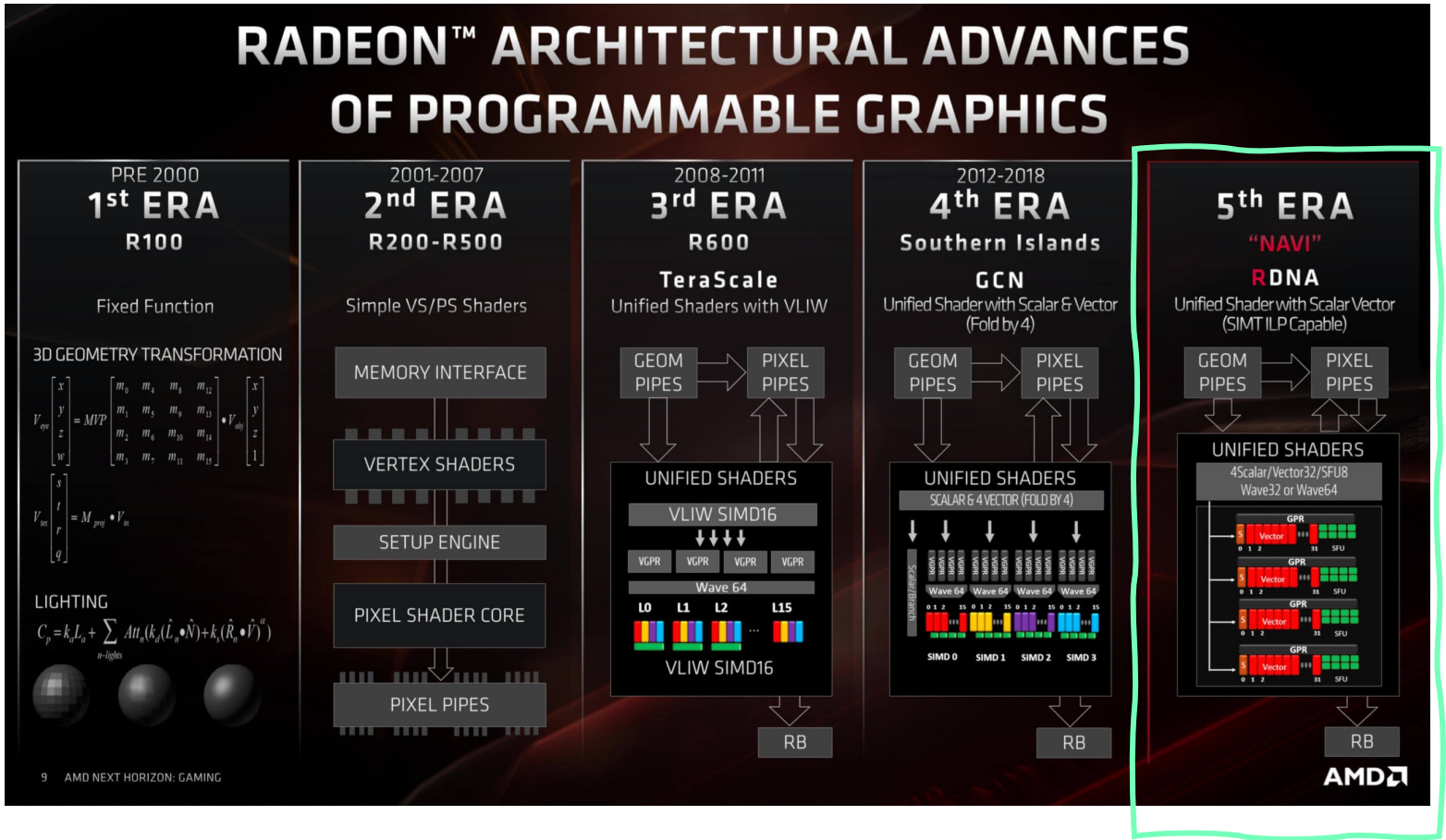
Arctehictural Evolution

- Southern Islands (2012)
  - The first generation of new GCN (Graphics Core Next) architecture
    - Radeon HD 7000-series: 2048 SPs from 32 CUs, 3.79 Tflops (FP32)
  - Fully general-purpose computing
  - Independent threads dynamically scheduled by the hardware in runtime
- Polaris (Gen 4; 2016) and Vega (Gen 5; 2017)
  - FP16 compute capabilities, improved power consumption, HBM2, instruction prefetching, etc.

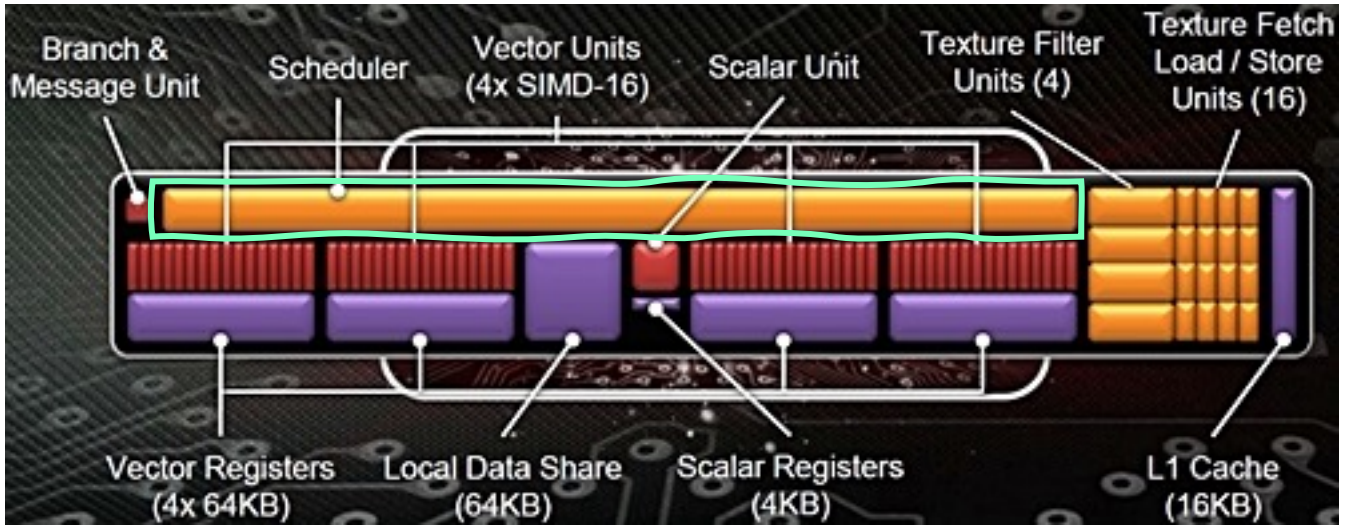


Arctehictural Evolution

- Navi (2019)
  - The first generation of new RDNA (Radeon DNA) architecture
    - Radeon RX 5700-series
  - Keeps the basic GCN architecture, reorganizes the data flow for performance
    - Narrower wavefront (64 → 32 work-items), wider SIMD → single-cycle instruction issue
    - Dual compute units (Workgroup processor)
- RDNA 2 (2020) and RDNA 3 (2022)
  - Increase in clock speed, memory hierarchy updates, ray tracing accelerator



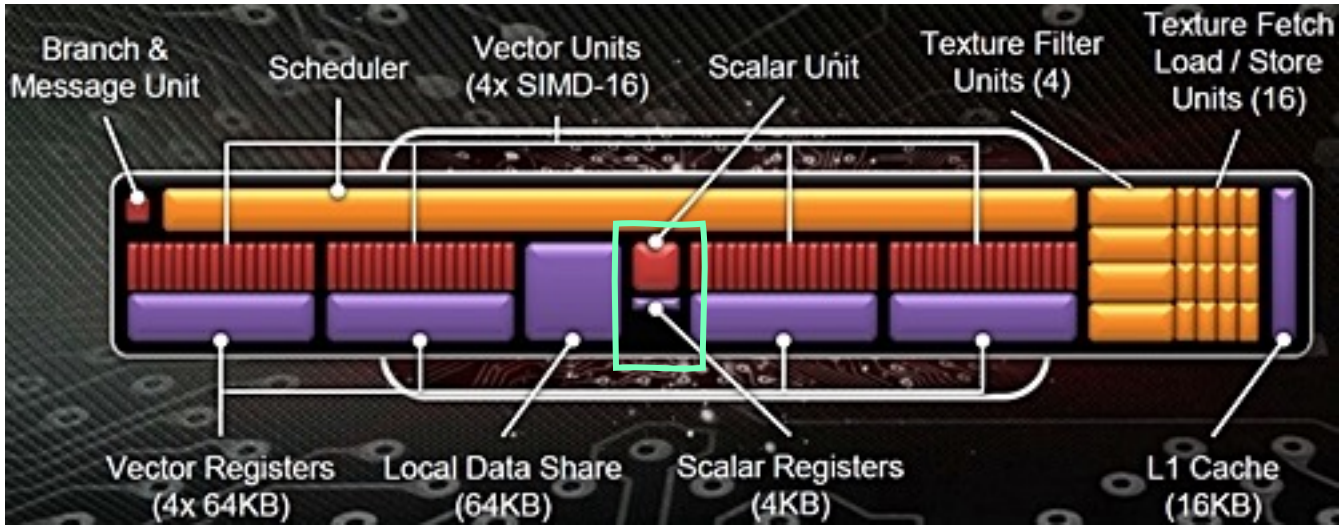
GCN: Compute Unit



◎ The command processor sends workgroups (as in OpenCL) to the compute units (CUs) through SE workload managers

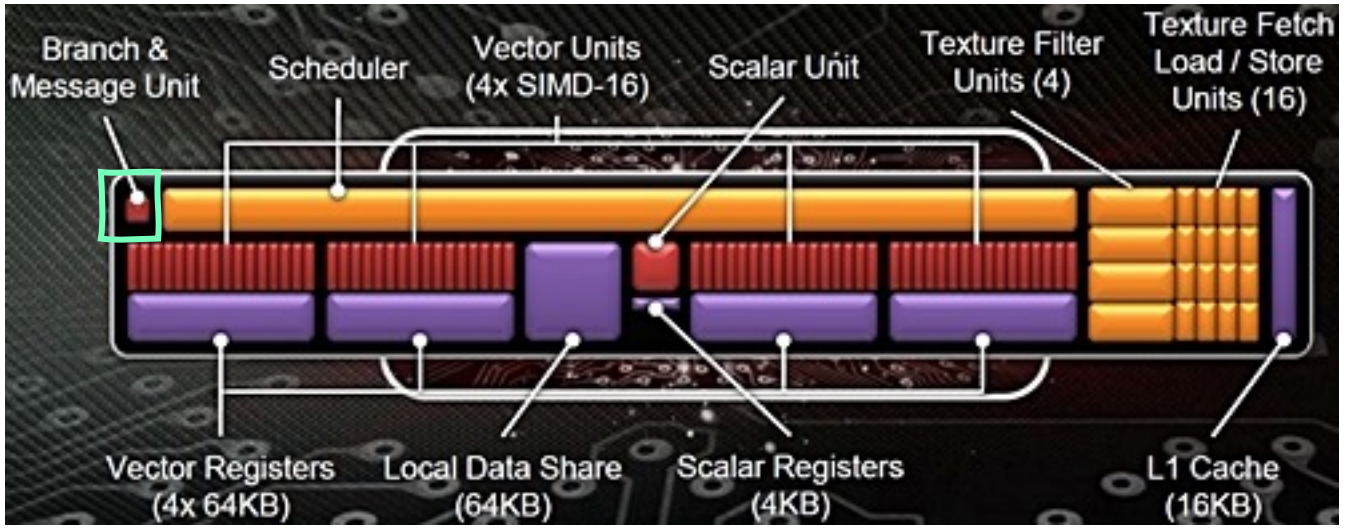
- Workgroups are executed in wavefronts (groups of 64 work-items)
- All wavefronts in a workgroup reside on the same CU
- The CU's scheduler can hold wavefronts from many workgroups
  - At most 40 wavefronts total per CU

GCN: Compute Unit



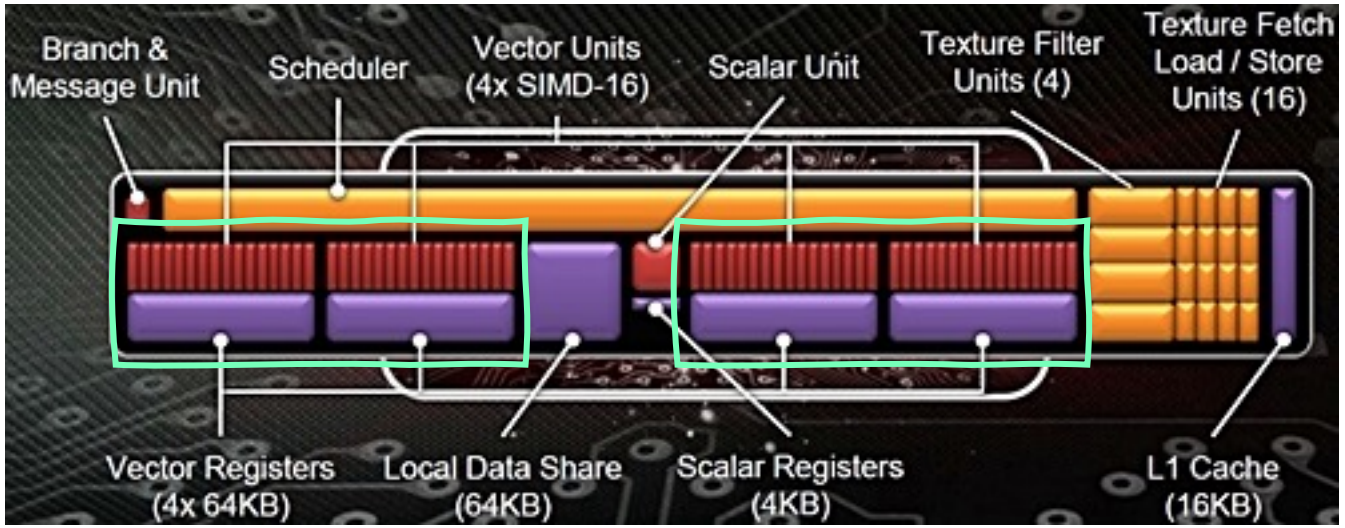
- Scalar Unit
  - Shared by all work-items in each wavefront
  - Work-items in a wavefront performing the exact same operation can offload the instruction to the SU
  - Loop iterators, constant variable accesses, branches, pointer arithmetic, scalar mathematical and transcendental operations
  - Has own pool of scalar general-purpose register (SGPR) file

GCN: Compute Unit



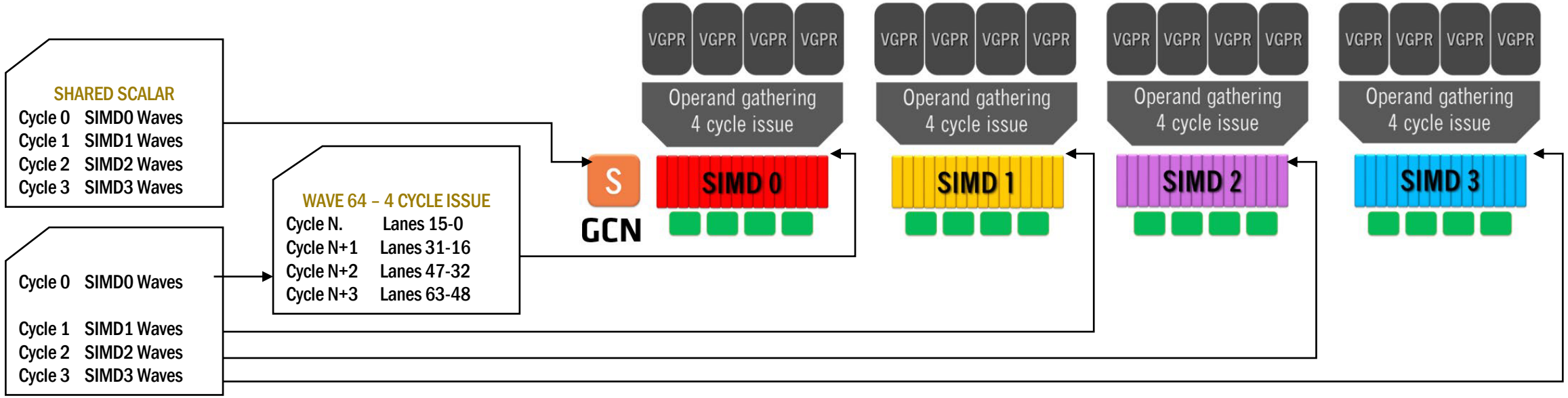
- Branch and Message Unit
  - Handle unconditional and conditional branches
  - Handle CPU interrupts with optional halt, debug messages, special graphics synchronization messages

GCN: Compute Unit



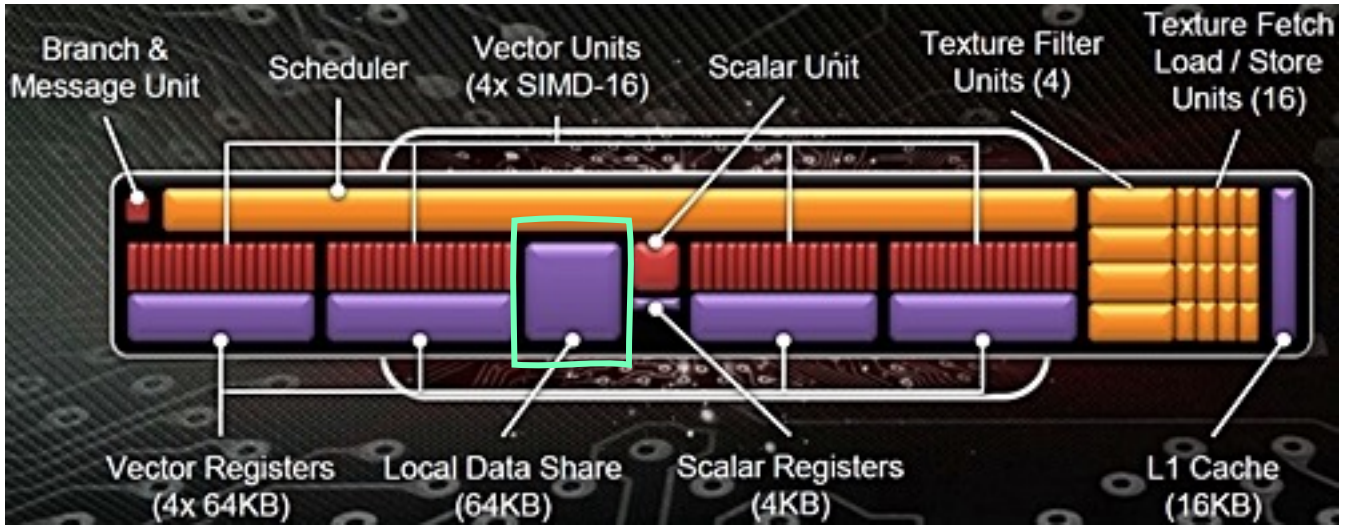
- Vector Units
  - 4x SIMD vector units; each SIMD has
    - A 16-lane vector ALU
    - 64KB vector general-purpose register (VGPR) file
    - Its own 40-bit program counter
    - Instruction buffer for 10 wavefronts: a wavefront = 64 threads

GCN: Compute Unit



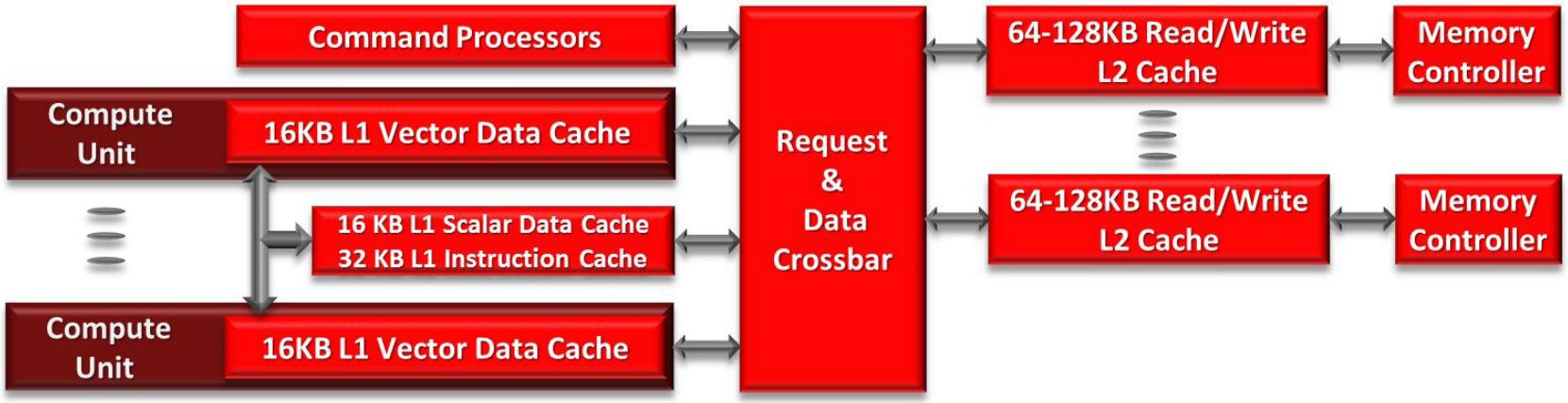
- At each cycle
  - 1 instruction per wavefront issued
  - 1 vector unit considered for execution (round-robin scheduling)
  - At most 1 instruction from different category can be issued (scalar, vector, memory, branch)
- A vector instruction takes 4 cycles
- Each vector unit can work on different wavefronts
- Maximum of 5 instructions can be issued every cycle (simultaneous instructions must come from different wavefronts)

GCN: Compute Unit



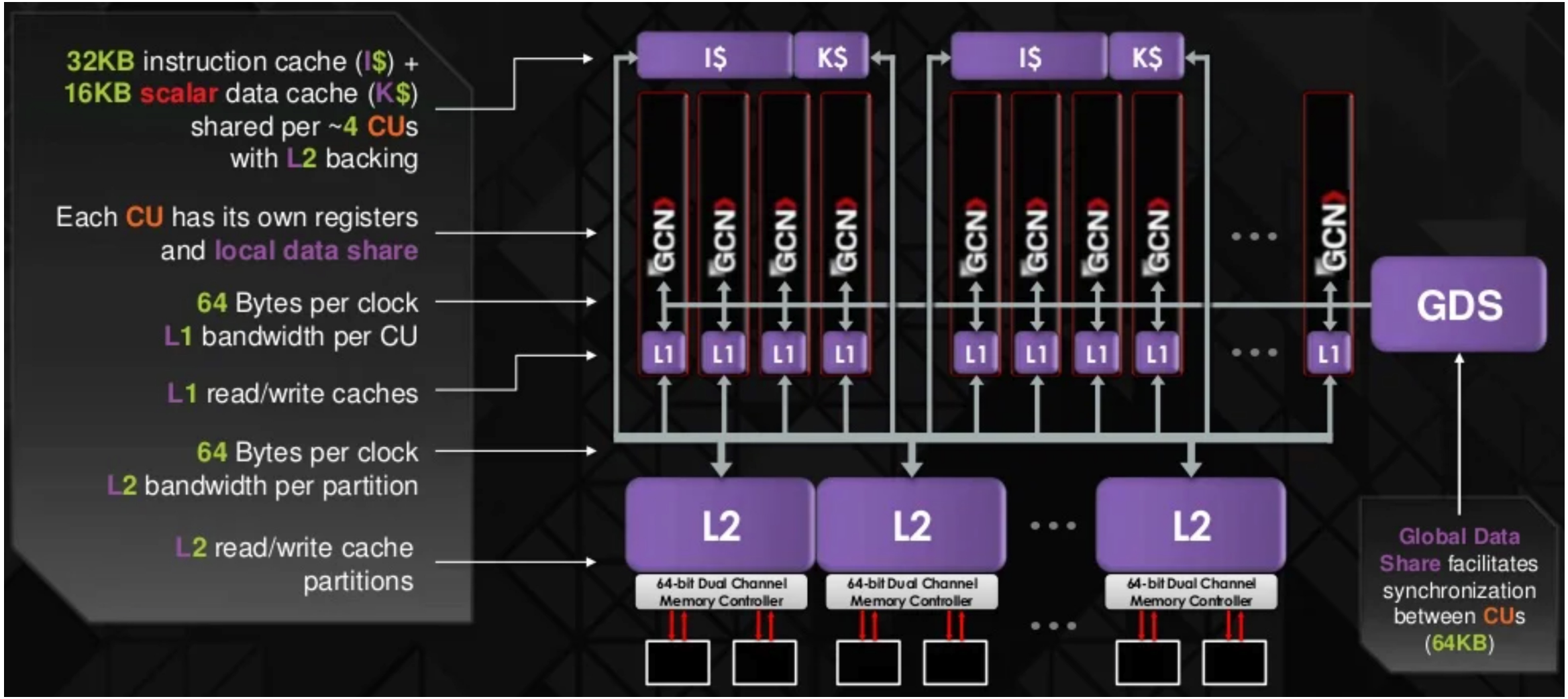
- Local data share (LDS)
  - 64KB scratchpad memory
  - 32 banks, with conflict resolution
  - Can be used to shared data between all work-items within a workgroup
  - Explicitly addressed memory that acts as a third register file

GCN: Cache Hierarchy



- Read/write L1 caches
  - 16 KB per CU
  - Write-through
  - 64B cache lines, 4-way associative, write-through
  - Relaxed consistency model: cache lines are written back to the L2 cache at thend end of a wavefront
- Read/write L2 caches
  - Distributed and coupled with each memory channel
  - 64B cache lines, 16-way associative, write-back
  - Supports atomic operations and synchronization between different wavefronts

GCN: Cache Hierarchy



AMD RDNA (Radeon DNA) Architecture

◎ Optimized for efficiency and programmability while backward compatible with the GCN architecture

◎ What's new?

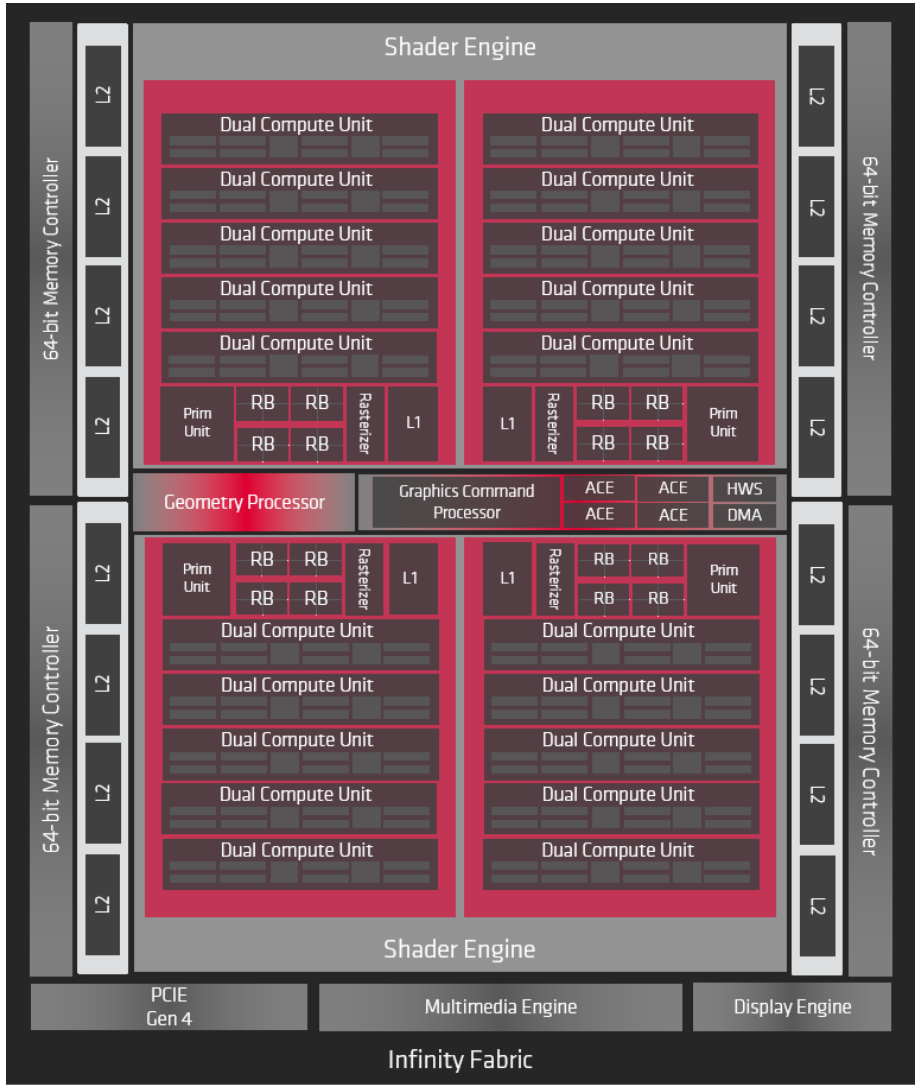
1. Updates to SIMD and CU
2. Addition of a new cache layer

→ Improvement to performance and power efficiency

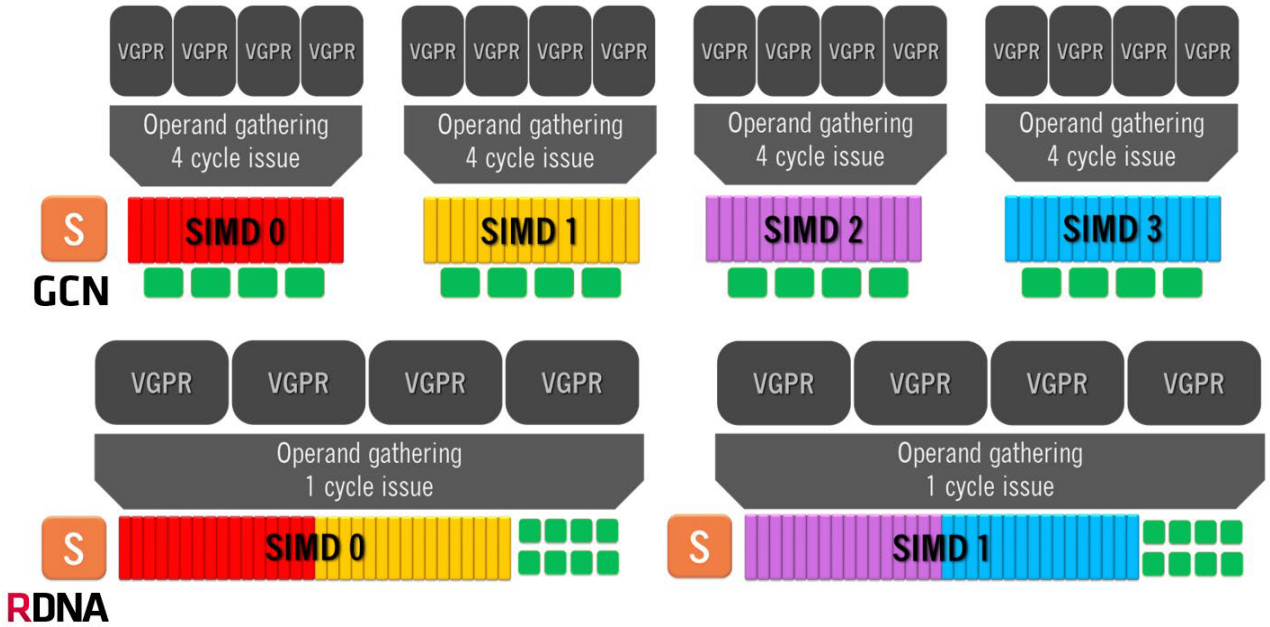
→ Target high-end gaming applications

◎ RDNA vs. CDNA

- RDNA for graphic processing and consumer applications
- PlayStation 5, Xbox Series
- CDNA for data-center applications (AI, HPC)

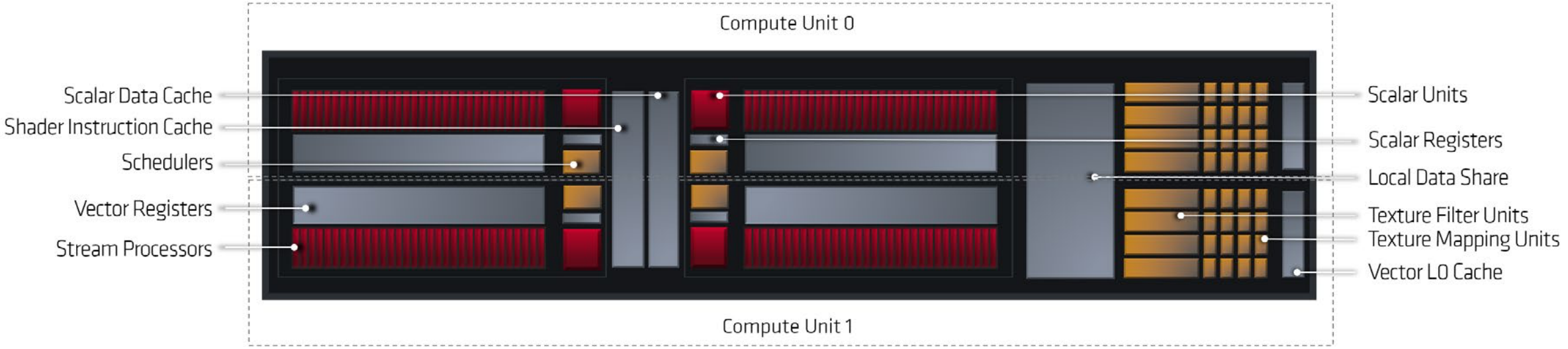


RDNA: SIMD Units



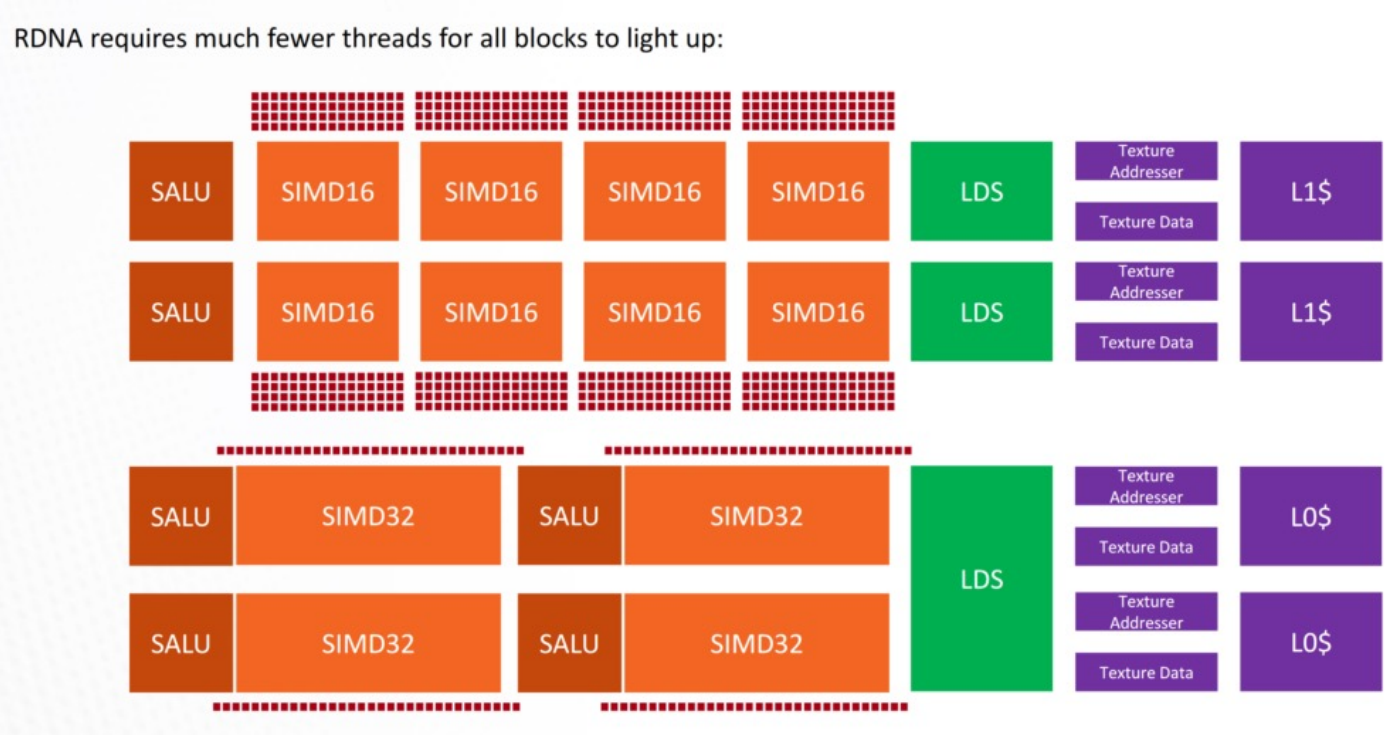
- 32-wide SIMD engines (twice as wide as GCN)
  - Supports mixed-precision operation
- 32 work-item wavefronts (half as many as GCN)
  - Handles complex control flow better (loops, function calls, branches)
  - Lower register pressure, enabling more active wavefronts → more efficiency latency hiding
  - Increase the total number of wavefronts → higher parallelism

RDNA: WGP (Work Group Processor)



- WGP (dual compute unit) is now the basic buiding block in RDNA
  - Two SIMD32 units per CU
  - Each SIMD unit has its own scheduler (c.f. a single scheduler per CU)
  - Two CU's per WGP sharing LDS, scalar data cache, and instruction cache

RDNA: WGP

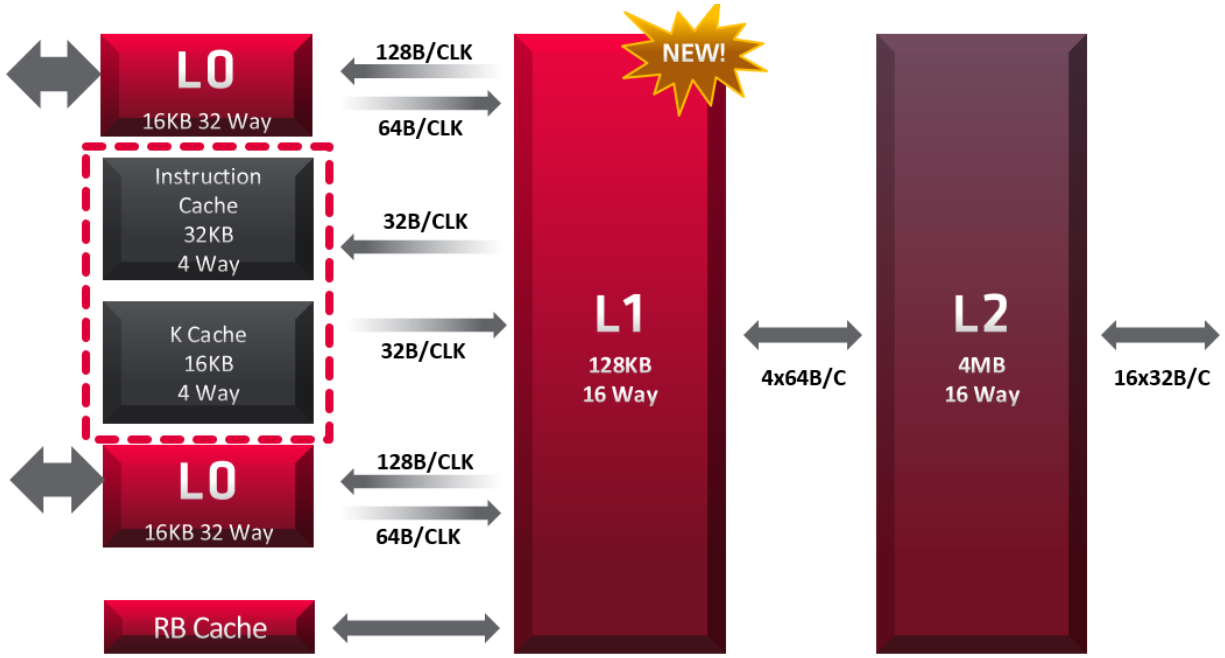


◎ Matching SIMD engine width and wavefront size

→ Execute a whole wavefront in one clock cycle

→ 4x speedup in wavefront execution time

RDNA: Cache Hierarchy

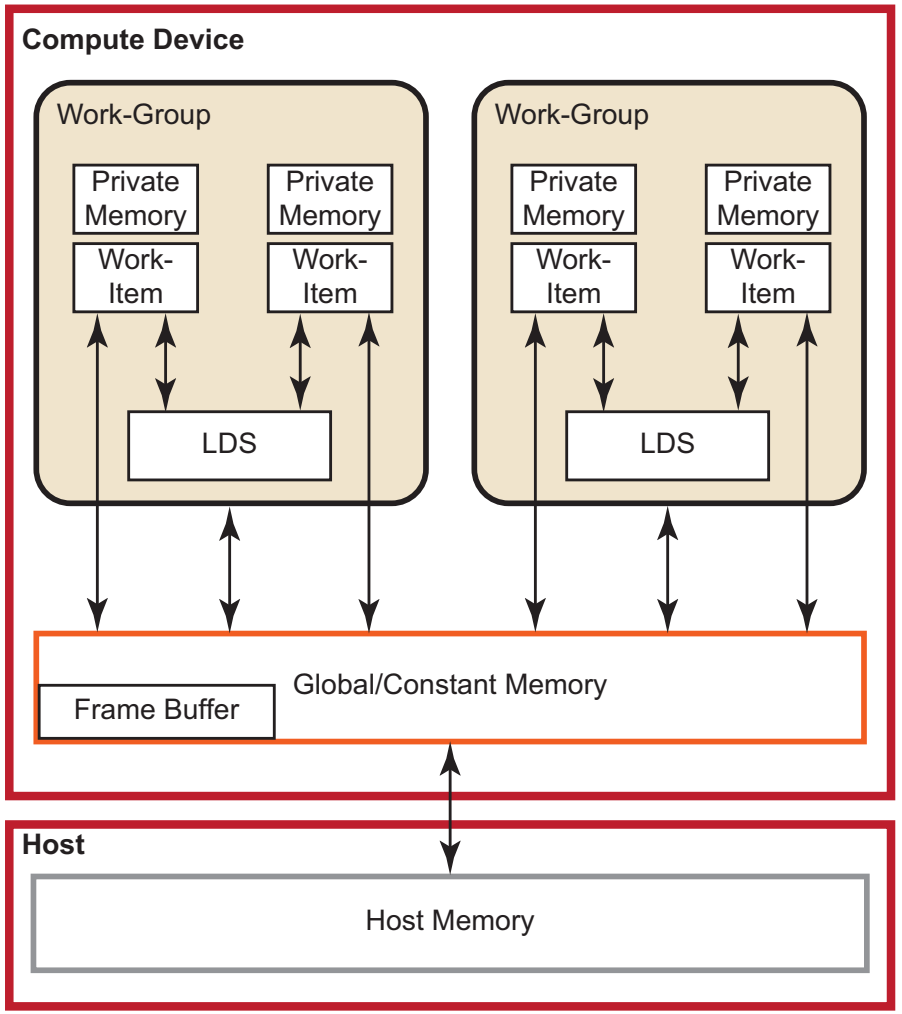


L1 Cache Hierarchy

- A new intermediate caching layer introduced to keep up with single-cycle wavefront execution
  - L1 caches in GCN → L0 caches (name changes)
  - New read-only L1 caches shared across 10 CU's
  - L1 caches interact with the global L2 cache

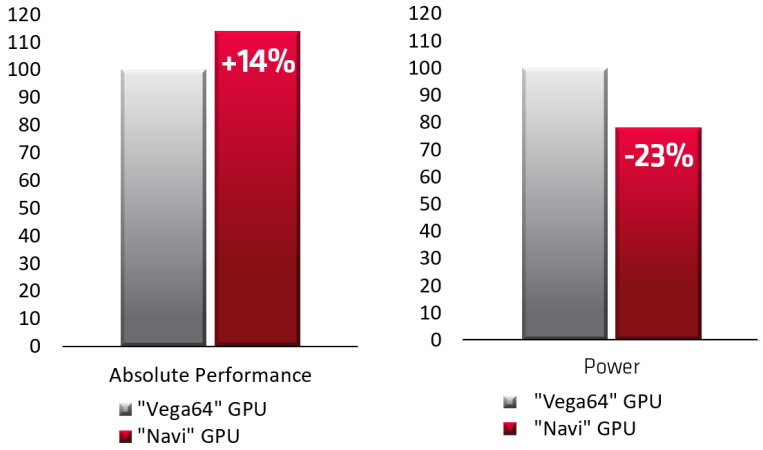
AMD Platform and Memory Hierarchy in OpenCL

- Literal mapping
  - Platform
    - Device, Compute unit, work-groups, work-items
    - Processing element → SIMD lanes in vector units
  - Memory hierarchy
    - Global, constant, private memory
    - Local memory → LDS (local data share)

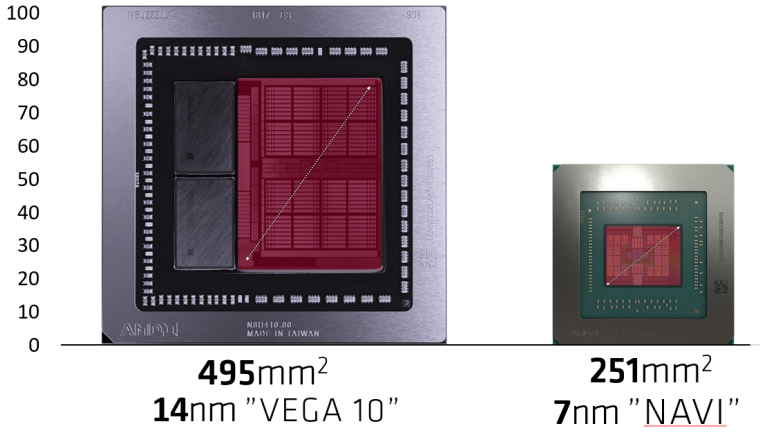


RDNA: Performance and Power Efficiency

1.5X PERFORMANCE PER WATT



2.3X PERFORMANCE PER AREA



495mm² **14**nm "VEGA 10"

251mm² **7**nm "NAVI"

- 7nm process gain
- Performance per clock enhancements
  - Single-cycle wavefront execution
- Cache hierarchy with higher bandwidth
  - Saves power and enhances scalability by reducing global L2 cache and memory requests

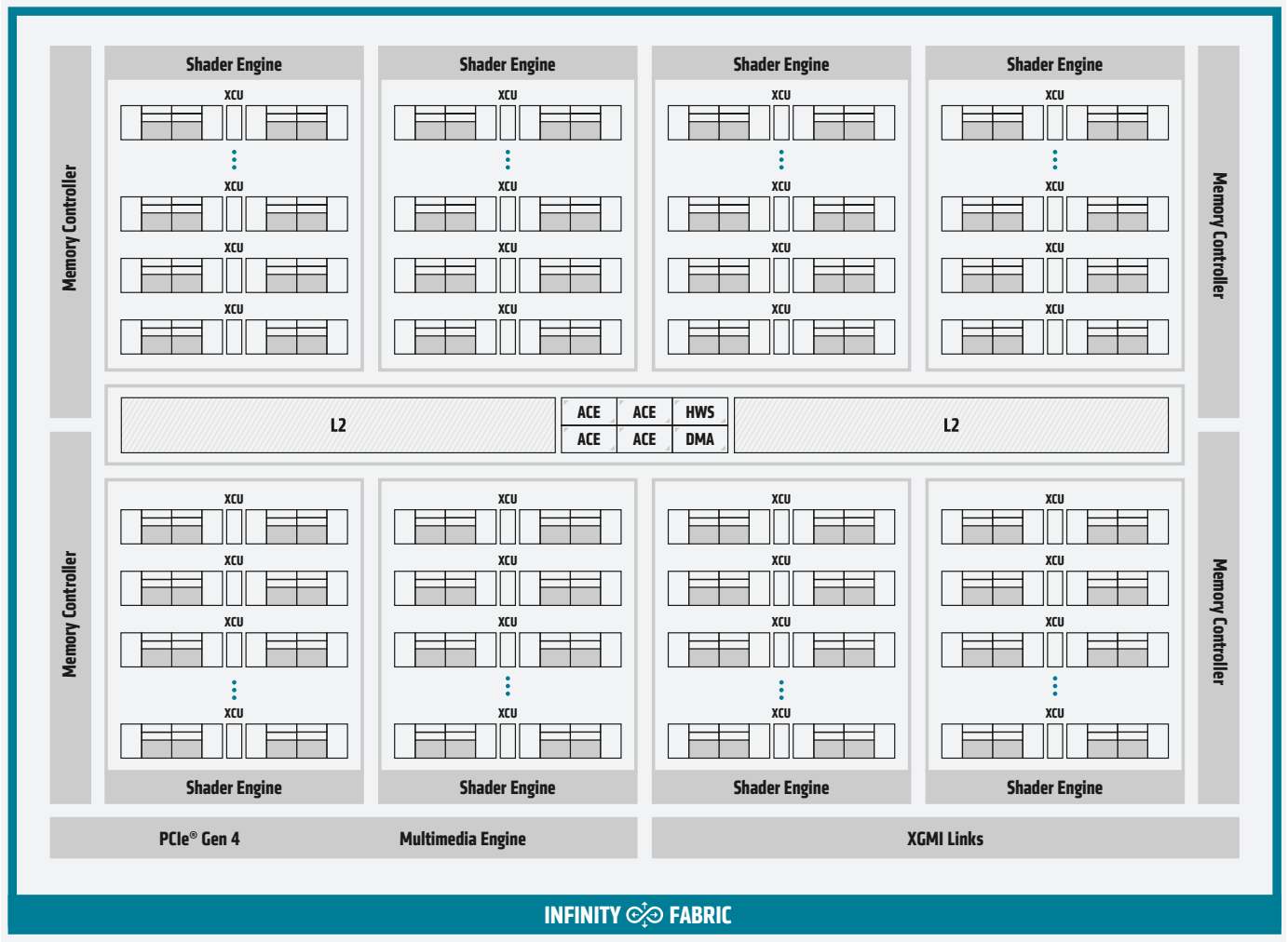
Recent RDNA and CDNA Developments

- RDNA 2 (2020): Improved memory efficiency and hardware ray tracing
  - Infinity Cache: a larger LLC for higher data reuse
- RDNA 3 (2022): WMMA matrix instructions
  - Wavefront-wide instructions for accelerating MMA
- RDNA 4 (2025): Expanded matrix acceleration

- CDNA 1 (2020): Wave64 compute and matrix cores
- CDNA 3 (2023): Integrates CPU, GPU, and HBM as chiplets
  - Have both discrete GPUs and shared physical HBM across CPU and GPU
- CDNA4 (2025): Support for HBM3E and expanded low-precision MMA acceleration

AMD CDNA (Compute DNA) Architecture

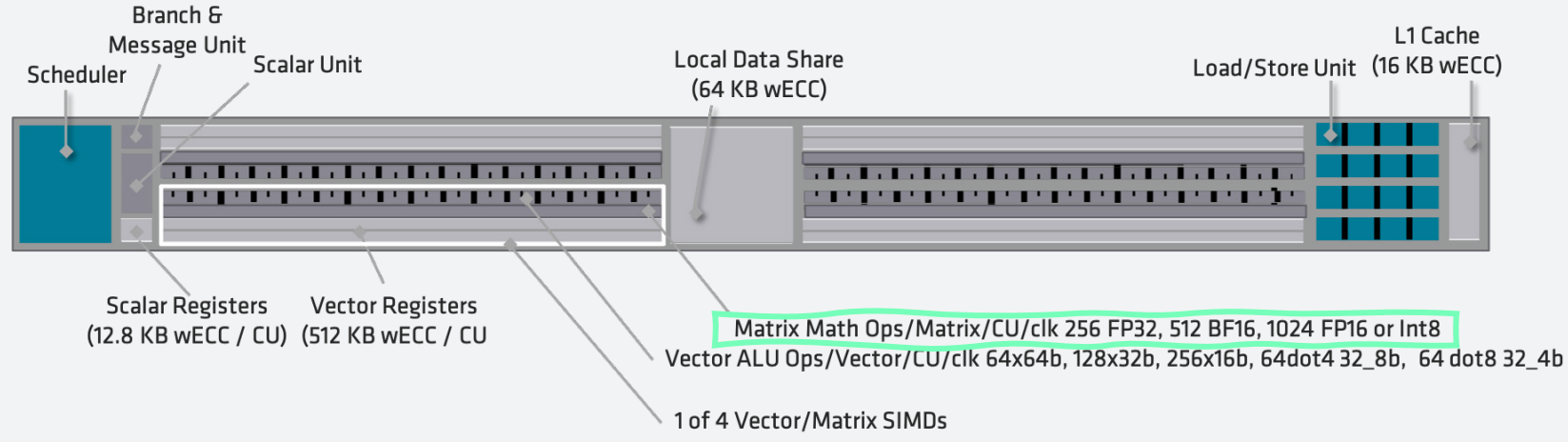
- 4 Asynchronous Compute Engines (ACE)
  - Maintains an independent stream of commands and can dispatch wavefronts to the compute units
- CUs based on the GCN architecture
  - 120 CUs organized into 4 arrays
  - Wavefront of 64 work-items
- AMD Infinity fabric
  - Connect to other GPUs (23 GT/s)
  - Supports coherent GPU memory, enables multiple GPUs to share an address space



https://www.amd.com/content/dam/amd/en/documents/instinct-business-docs/white-papers/amd-cdna2-white-paper.pdf

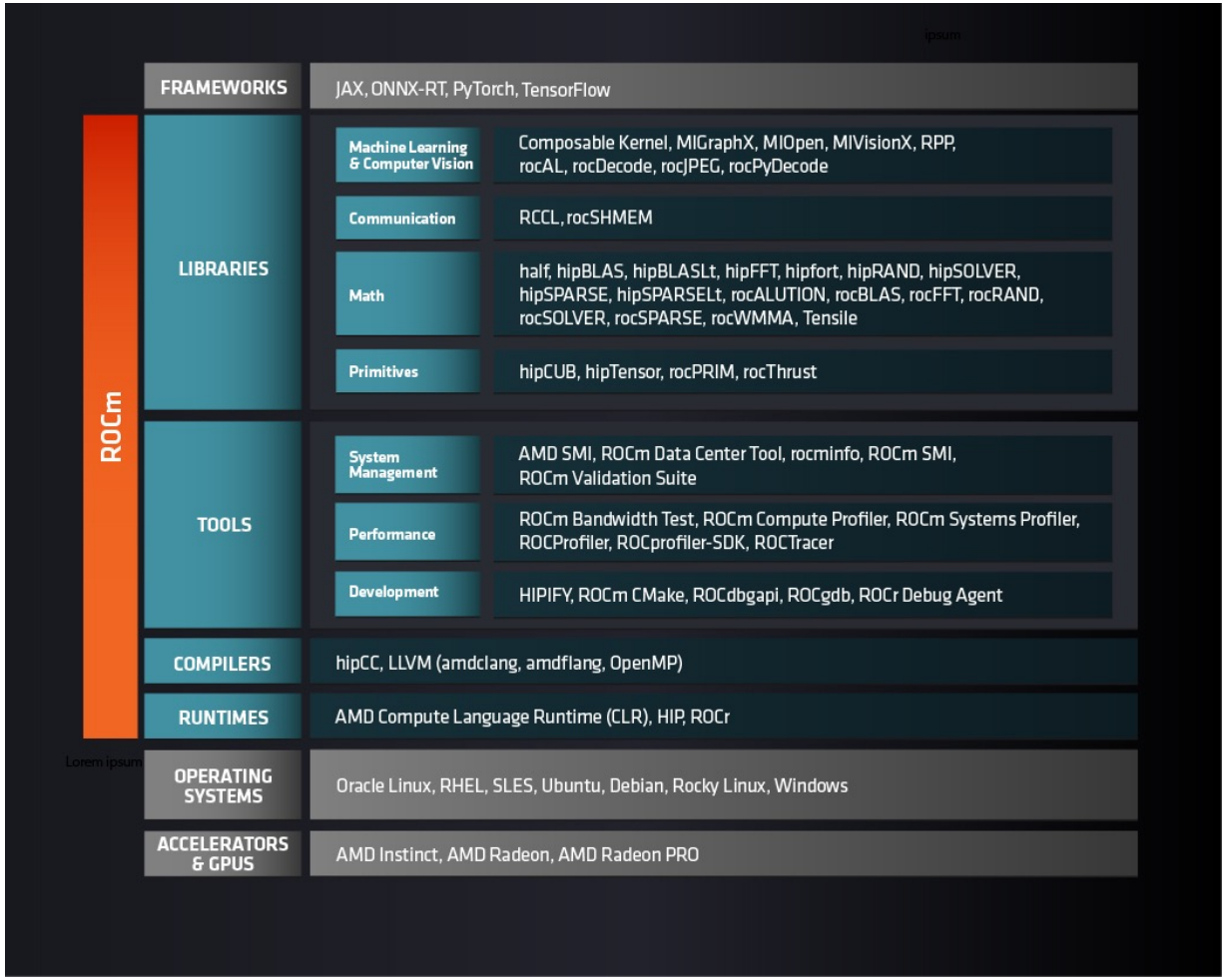
AMD CDNA Architecture

- Introduces matrix cores
  - Optimized for executing compute cores that operate on matrix data datatypes
  - Supports wavefront-level instructions, the Matrix Fused Multiply-Add or MFMA
  - Mixed-precision arithmetic (INT4, INT8, FP16, BF16, FP32 input data)



Software Stack for AMD GPUs

- ROCm
  - Open-source software for programming AMD GPUS
  - Provides tools for HIP, OpenCL, and OpenMP
    - HIP is AMD's CUDA (very similar)
    - HIPIFY assists source conversion
    - HIP bindings available for Python and Fortran
  - Provides compilers, libraries, debuggers, profilers, and runtimes

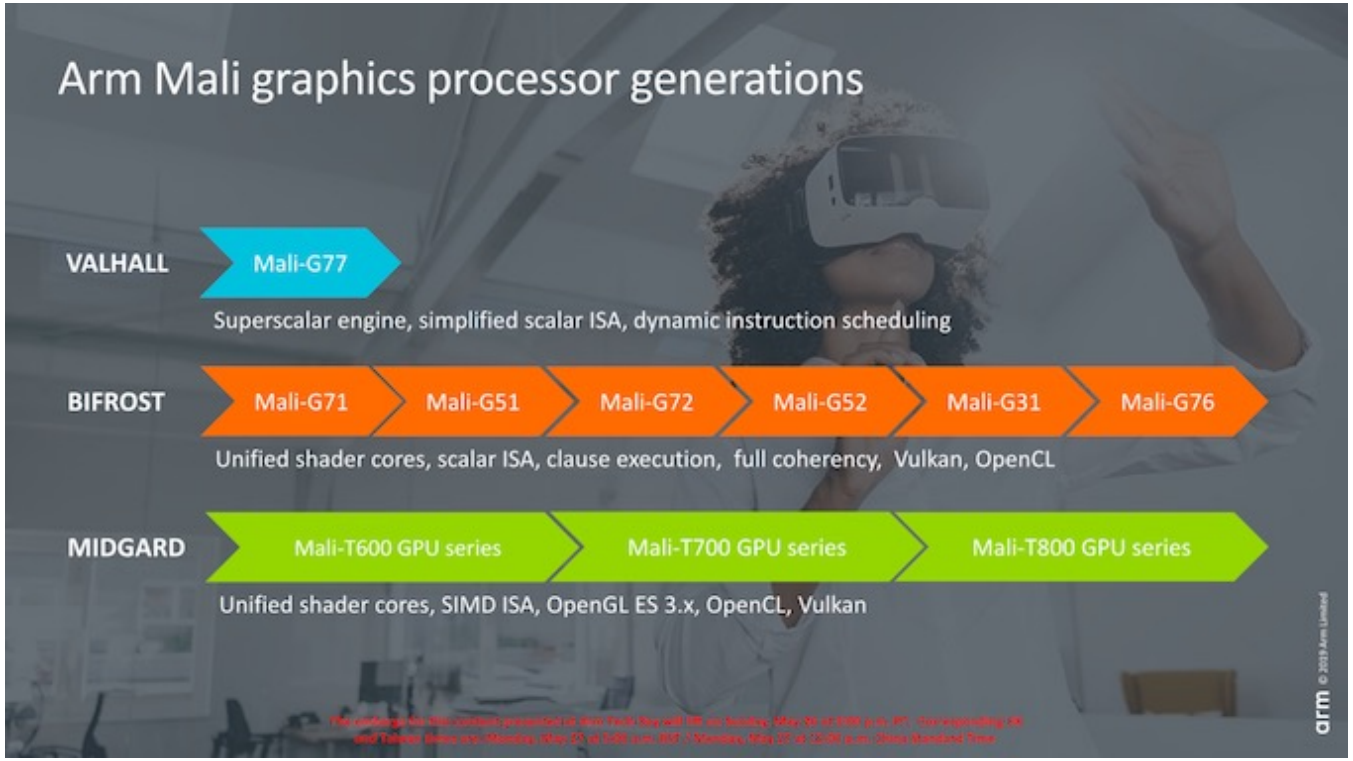


https://rocm.docs.amd.com/en/latest/what-is-rocm.html

ARM GPU Architectures

ARM Mali GPU

- Utgard (2006)
  - ARM acquired Falanx in 2006
  - Separate pixel and vertex shaders
- Midgard (2012)
  - Unified shaders
- Bifrost (2016)
  - Full coherency
  - Scalar instructions across grouped threads
- Valhall (2019)
  - Hardware scheduler
  - Wide warps and dynamic instruction scheduling
- 5th Gen (2023-)
  - Optimized graphics dataflow

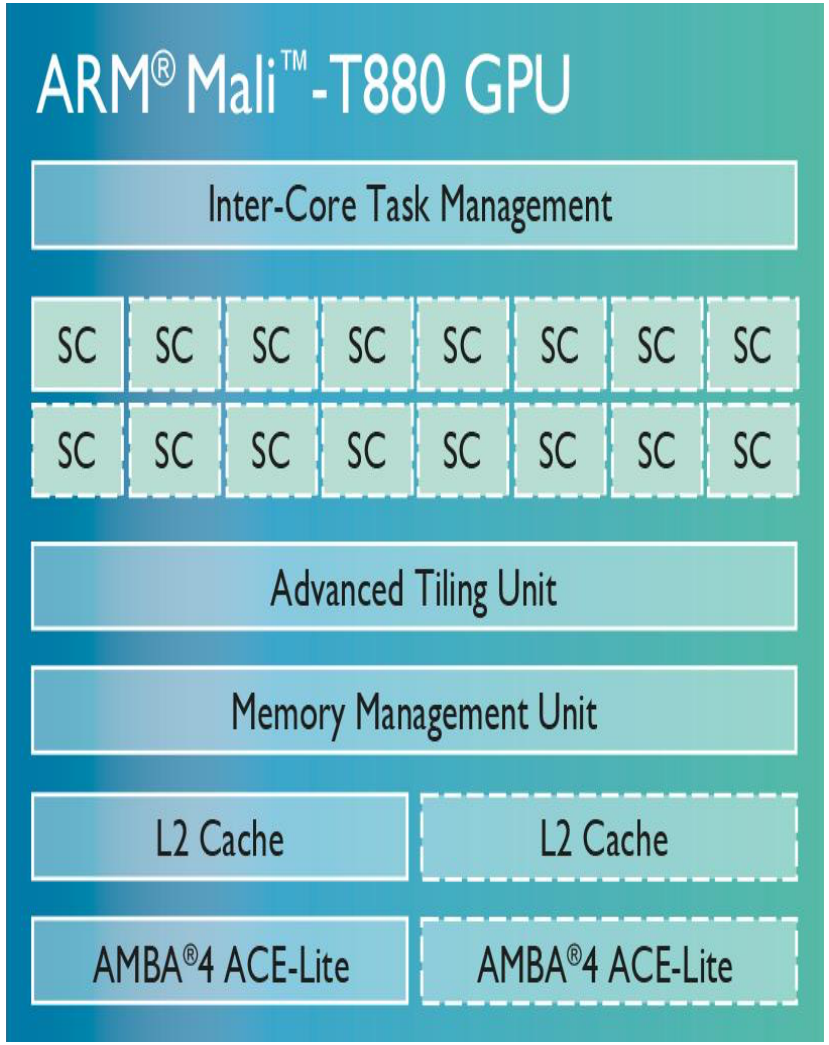


SoC (Systems-on-Chip) GPUs

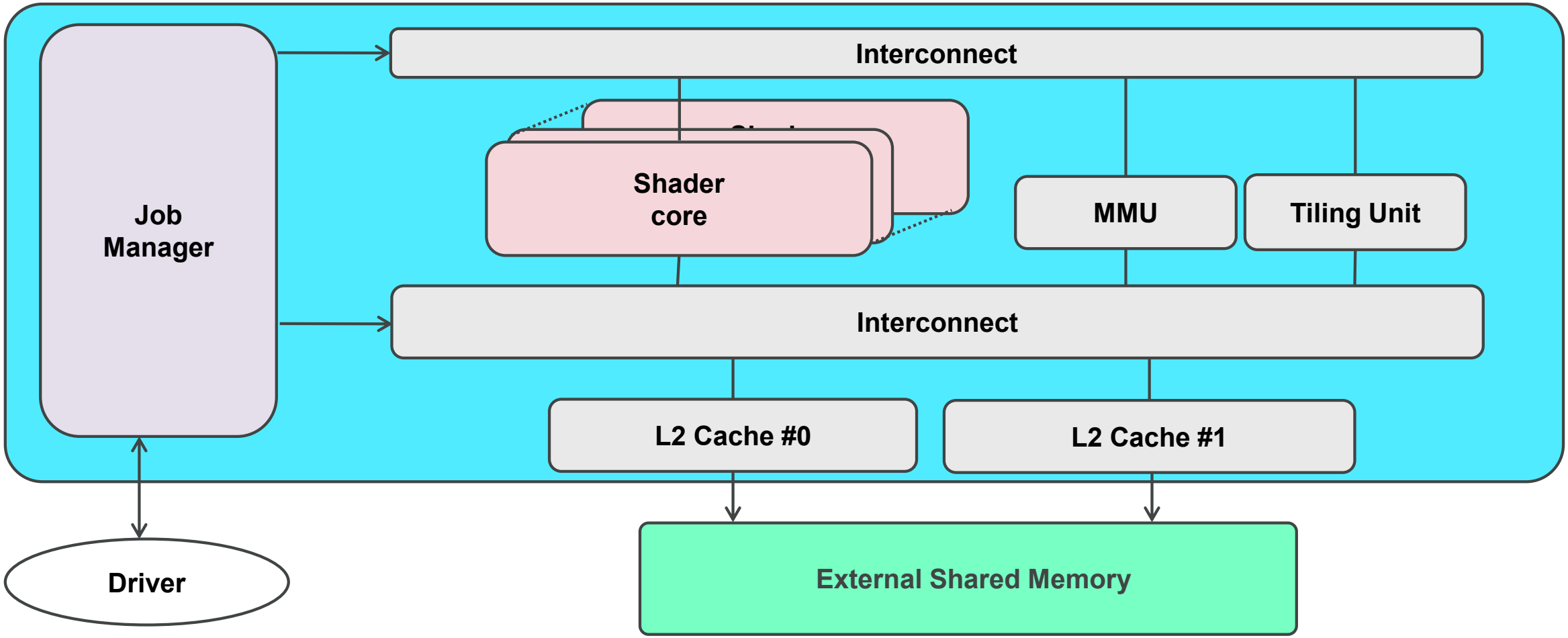
- Integrates most or all components of a computer on an IC
  - CPUs, GPUs, DSPs, memory interfaces, modems, etc.
- SoC GPU market divided between the vertically integrated Qualcomm and the IP-licensing ARM
  - Mid-range SoC GPUs on mobile and embedded devices
  - High-end SoC GPUs for gaming consoles
- Compared to desktop GPUs (NVIDIA, AMD)
  - Much more restricted in terms of power, area, and memory bandwidth
  - Focus more on accelerating graphic funtionalities
  - Design choices reflect mobile workloads and constratins

Midgard Architecture

- Tile-based, unified shading architecture
  - Maximum of 16 shader cores
  - Execute vertex, fragment, and compute programs
- Tile size is 16x16
- Supports OpenCL 1.2, OpenGL ES 3.2, Vulkan 1.0
- First product in 2011
  - Samsung Exynos 5250

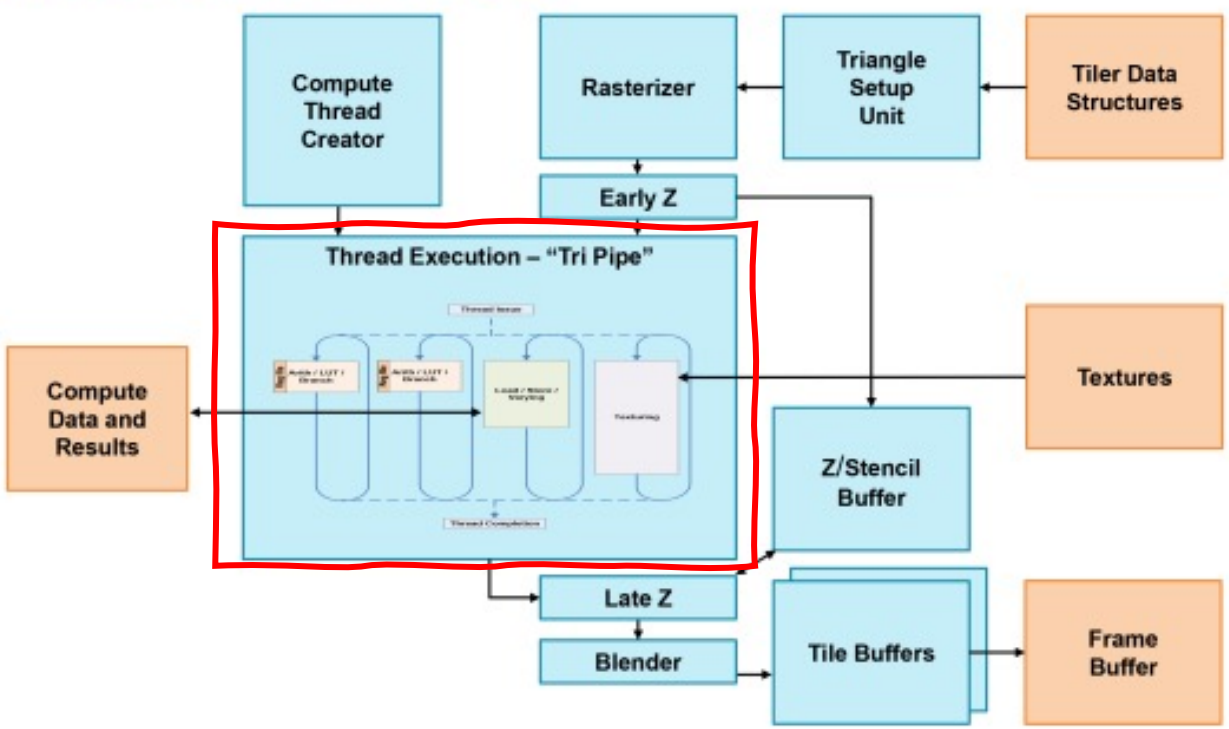


Architecture Overview



Shader Core

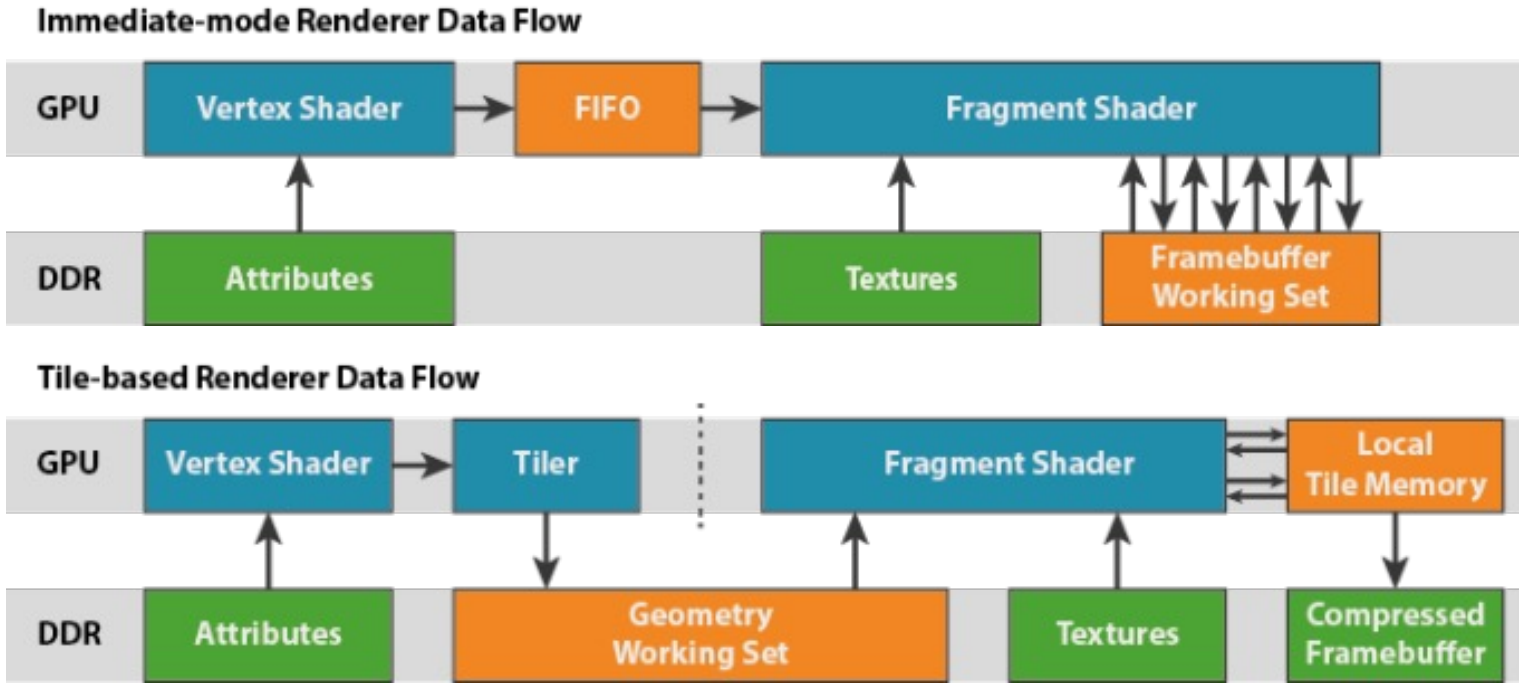
- "Tri Pipe"
  - 3 types of pipelines: arithmetic, load/store, and texturing
  - 128-bit operands, float or integer: 2xFP64, 4xFP32, 8xFP16



ARM

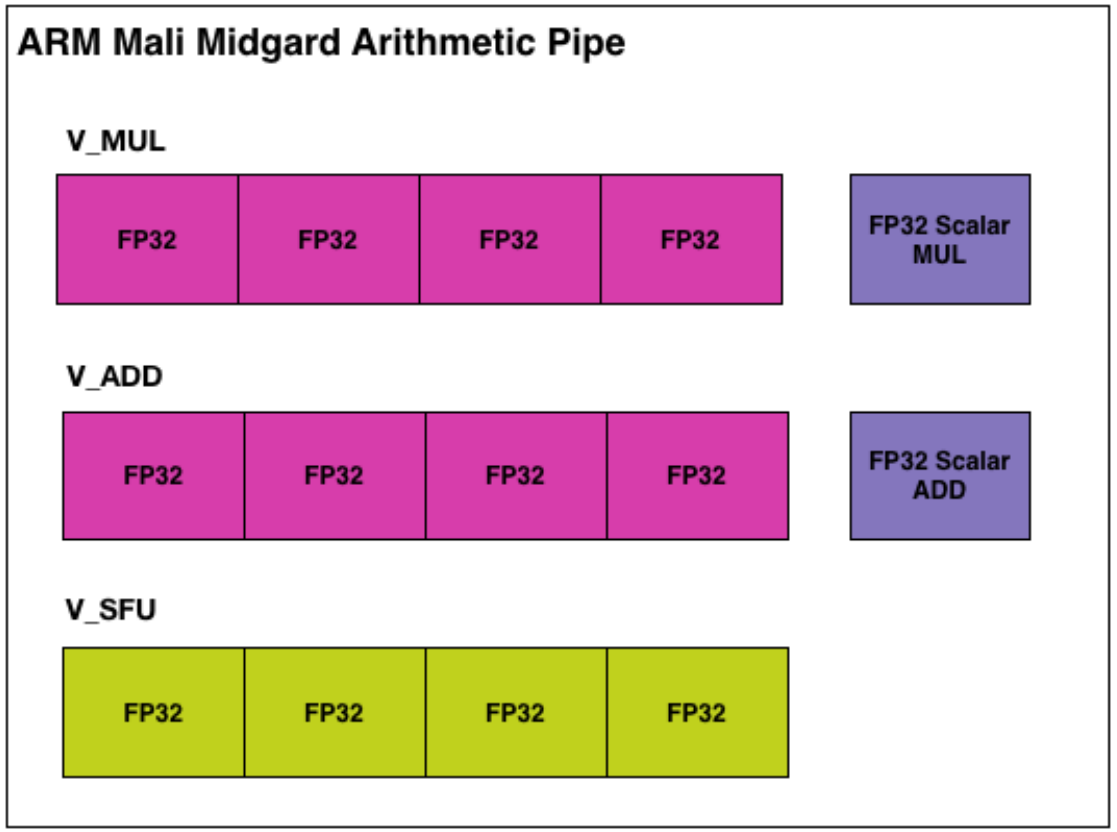
Tile-Based Rendering

- The tiling unit breaks up scenes into individual tiles to be processed on a tile-by-tile basis
- Fragment shader uses an on-chip tile buffer for intermediate results
- Reducing framebuffer traffic saves external bandwidth and energy
  - Keep a tile in local caches on the GPU, saving main memory accesses



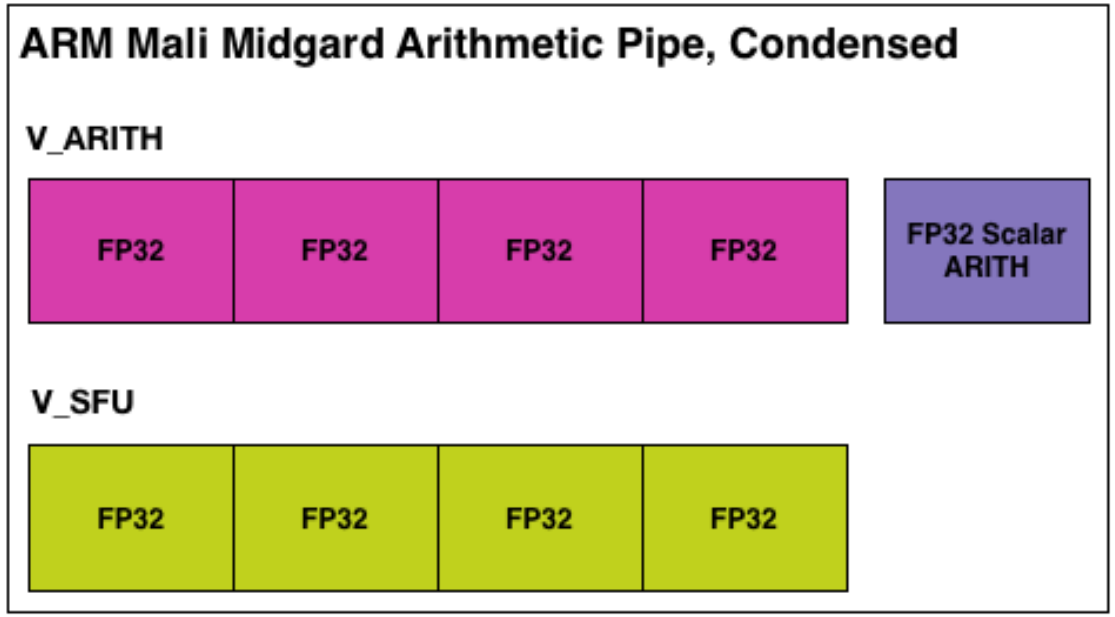
Arithmetic Pipeline

- VLIW
  - Sequential long instruction word target several arithmetic units
- A single arithmetic pipeline includes
  - A mix of both scalar and SIMD ALUs
  - Vector SFU for dot products, transcendentals, and other complex operations
- Instruction words contain instructions for multiple units
  - Compiler constructs a single word of instructions from a single thread to fill as many of the ALUs in an arithmetic pipeline as is possible

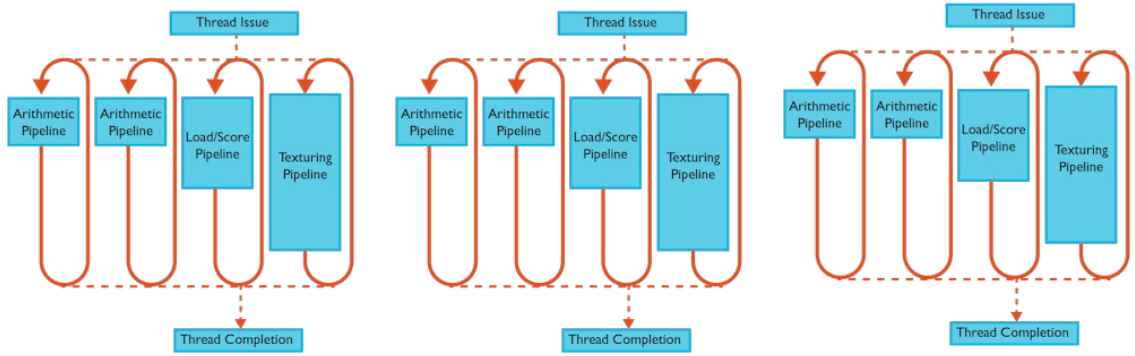


Midgard Execution Model

- SIMD
  - Vector instructions operate on multiple data elements packed in 128-bit vector registers

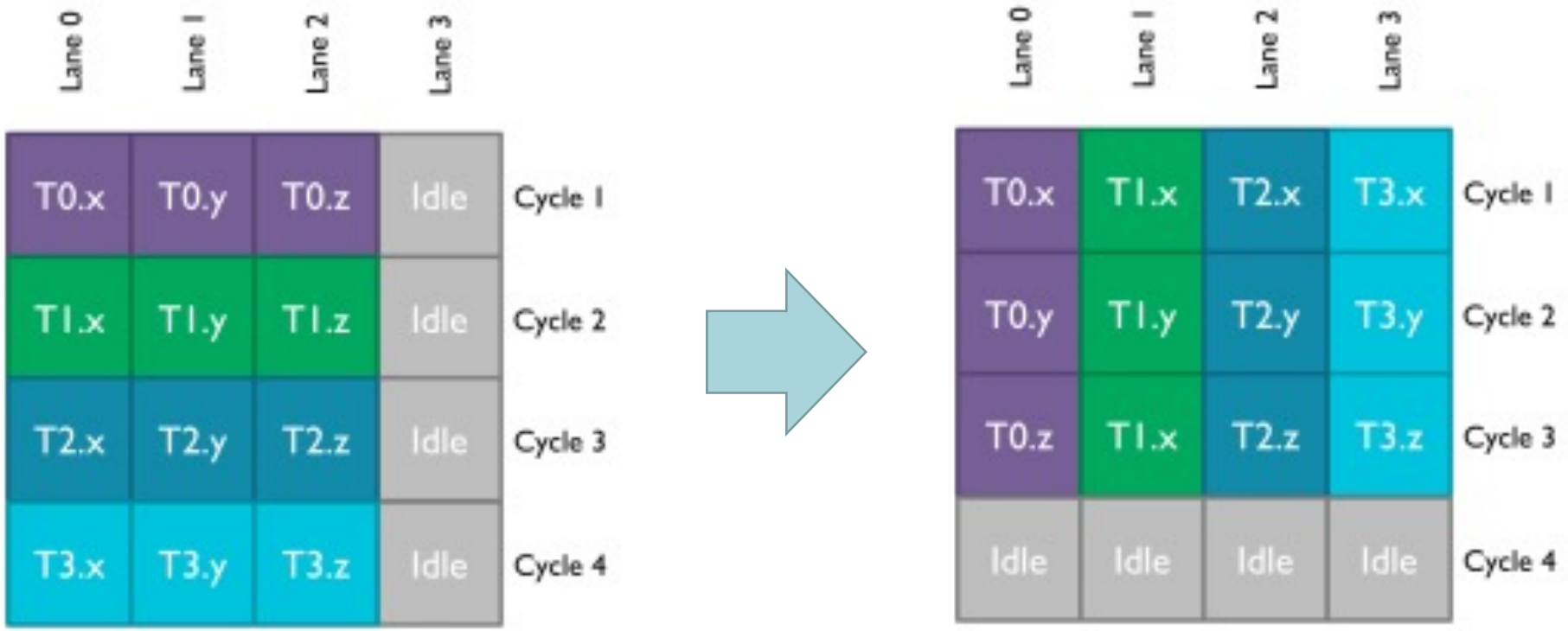


- FGMT (Fine-Grained Multithreading)
  - Each core runs its threads in a round-robin fashion, on every cycle switching to the next ready-to-execute thread



Bifrost Execution Model

- Scalar instructions from multiple threads fill the execution lanes
- TLP-centric execution model
  - Instead of extracting four instruction's worth of ILP from a single thread, groups 4 threads together and executes a single instruction from each one

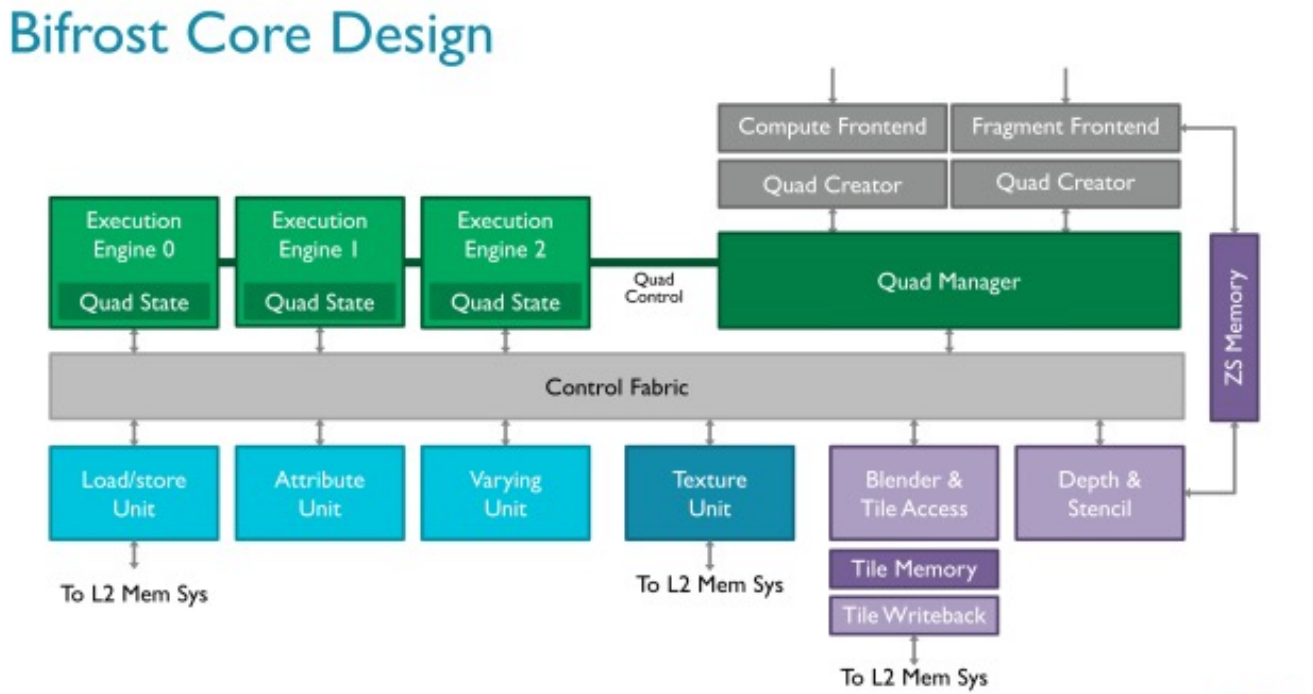


Quad Vectorization

- Limitations of the ILP+VLIW approach
  - Compiler vectorization is not perfect
  - Have to detect combinations of operations that can be merged to fill idle lanes
  - Scalar operations cannot always be merged into vectors
  - TLP provides better utilization, thus more power and area efficient
- Qual vectorization = qual-parallel execution
  - Four scalar threads executed in lockstep in a “quad”
    - C.f., warp/wavefront
  - One quad at a time executes in each pipeline stage
- Very similar transition to what AMD made with GCN

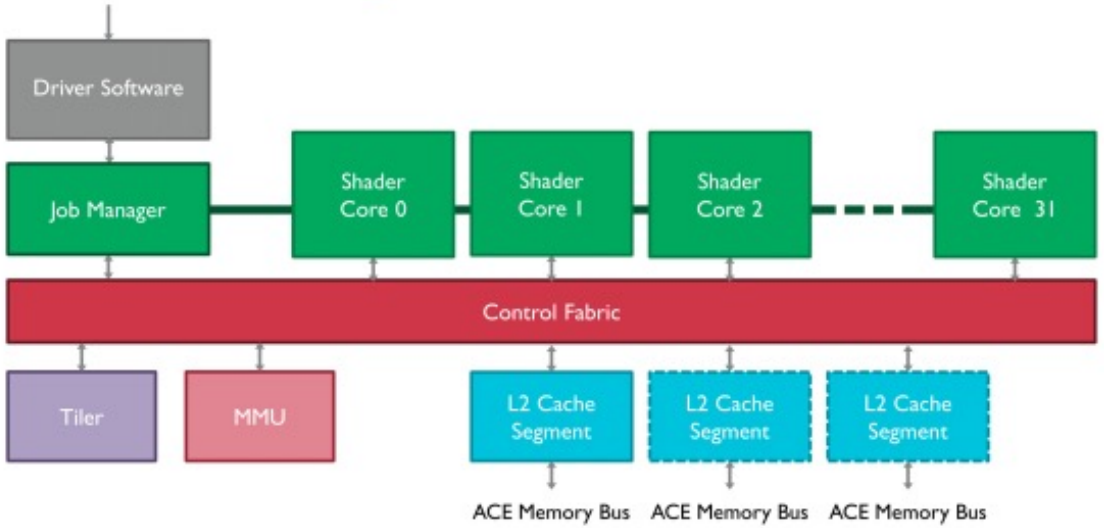
Bifrost Shader Core

- 3 quad-execution engines
- Quad manager: thread manager
- Load/store unit, texture unit, and other units as separate units from quad engines

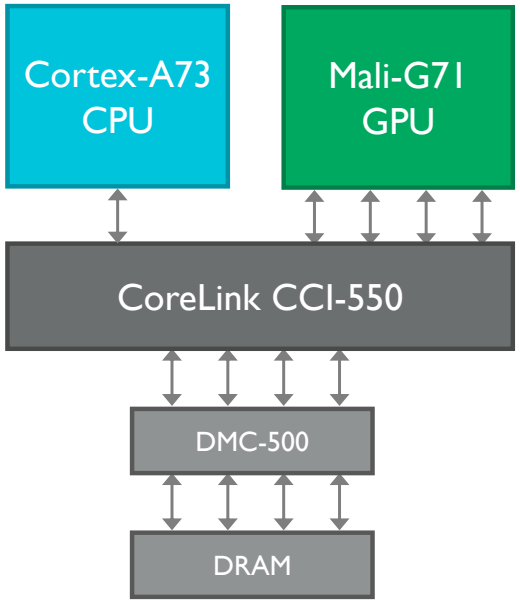


Bifrost Architecture

- Can include a variable number of shader cores (up to 32)
- Full coherence support
  - L2 cache is attached to the SoC coherent interconect (CoreLink CCI-500)
  - Supports tightly coupled CPU+GPU use cases

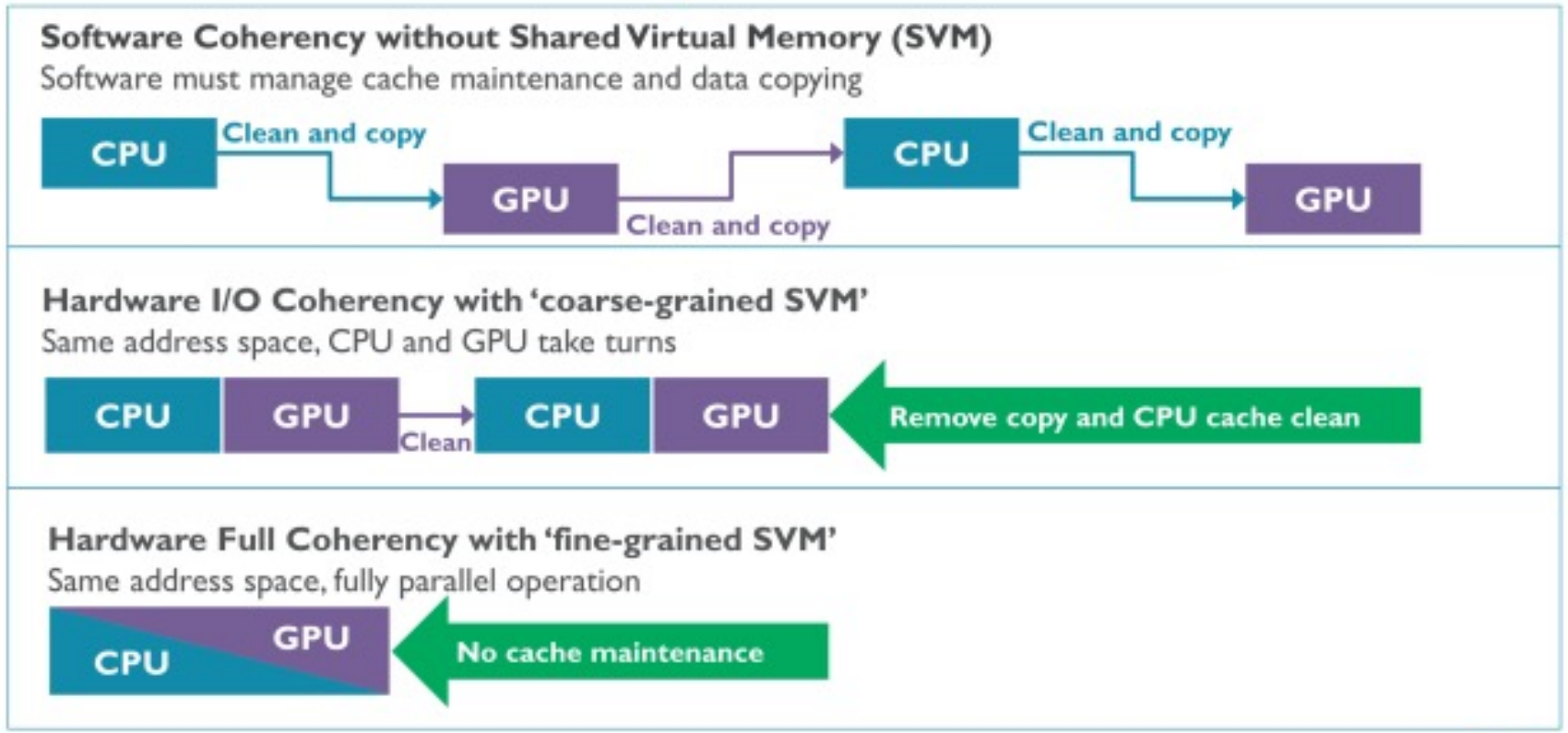


7 ©ARM2016
EMBARGOED UNTIL 11pm EDT on Sunday, May 29
ARM



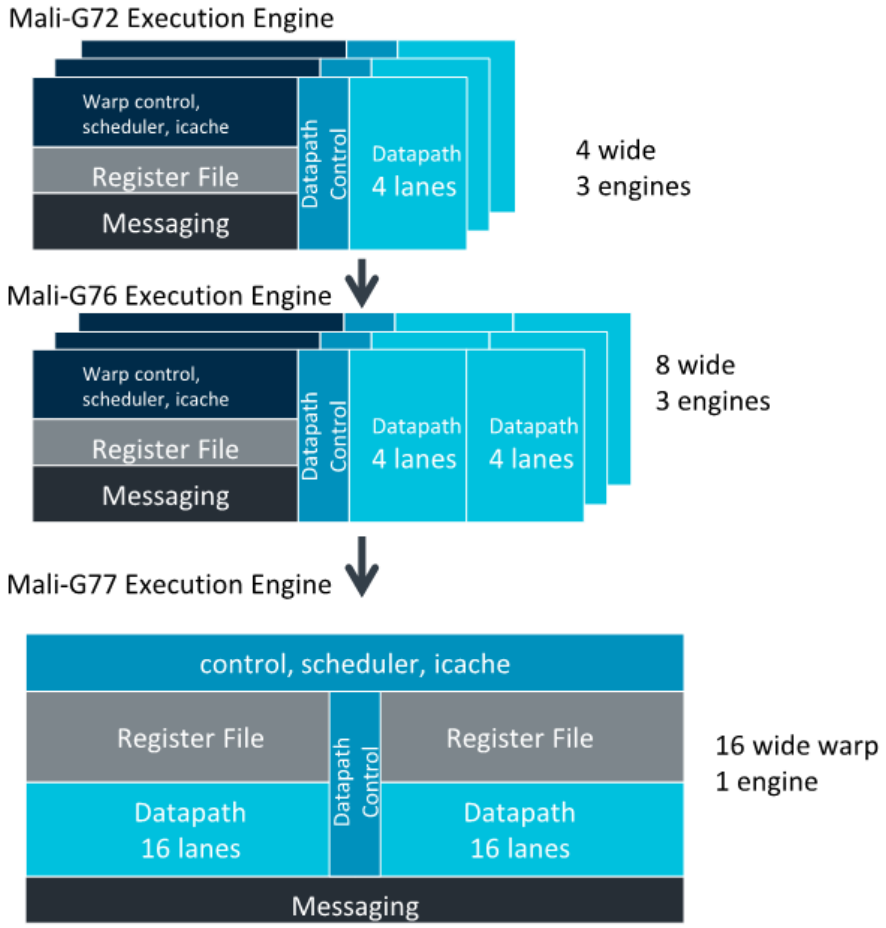
Shared Virtual Memory Removes Overheads

- Supports OpenCL 2.0 with shared virtual memory and related APIs



Valhall Architecture Goals

- 2nd generation of ARM GPU scalar architecture for high-performance, high-efficient GPUs
- 16-wide warp-based execution model
- New simplified and compiler-friendly instruction set



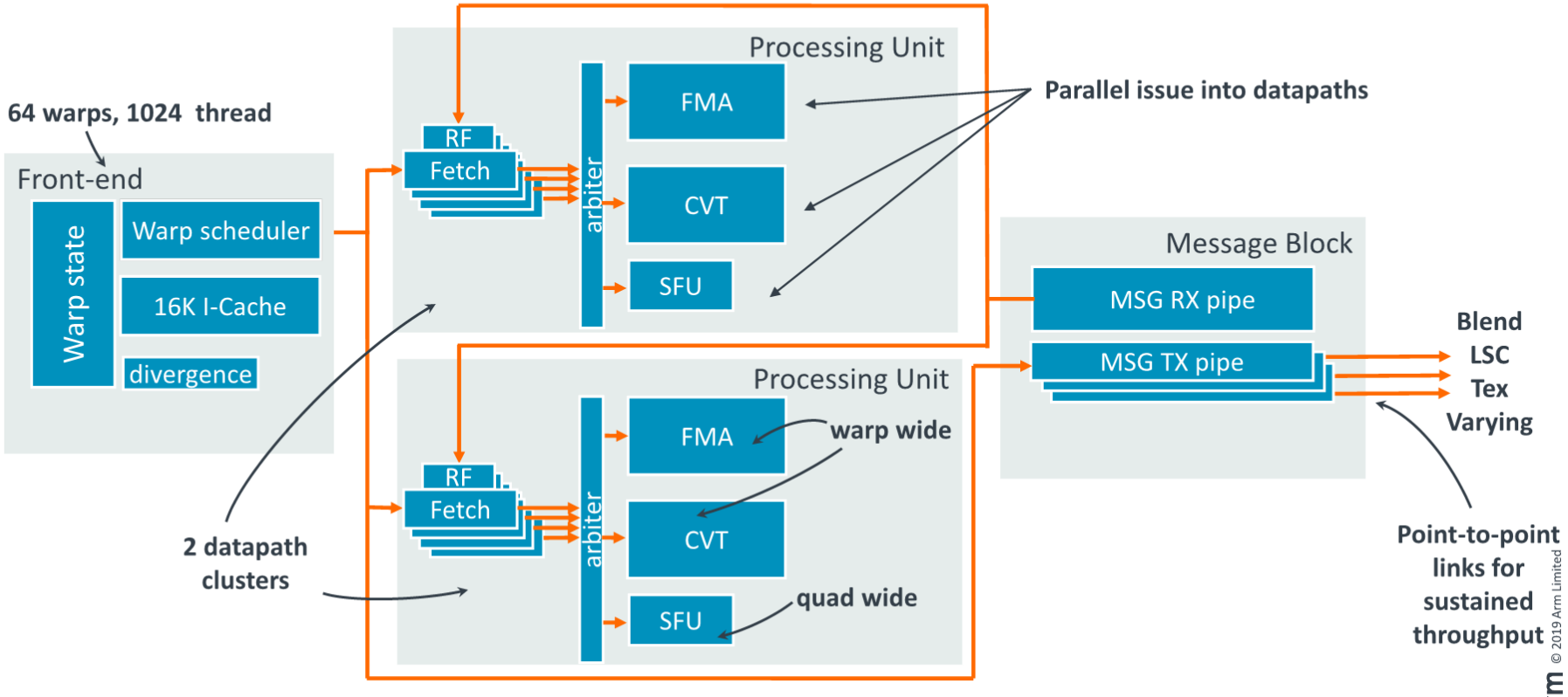
Valhall Architeture

- Warp-based execution model
  - 16 threads executed in lockstep in a warp
- New instruction set
  - Operational equivalence to Bifrost
  - Regular, unconstrained instruction encoding
- Multiple execution engines consolidated to a single CPU with shared control logic
  - Reduces overheads with datapath control logic, scheduler, i-caches, and registers
- Dynamic scheduling of instructions
  - Done by HW
  - No more clauses, tuples and fixed issuing



Valhall Microarchitecture

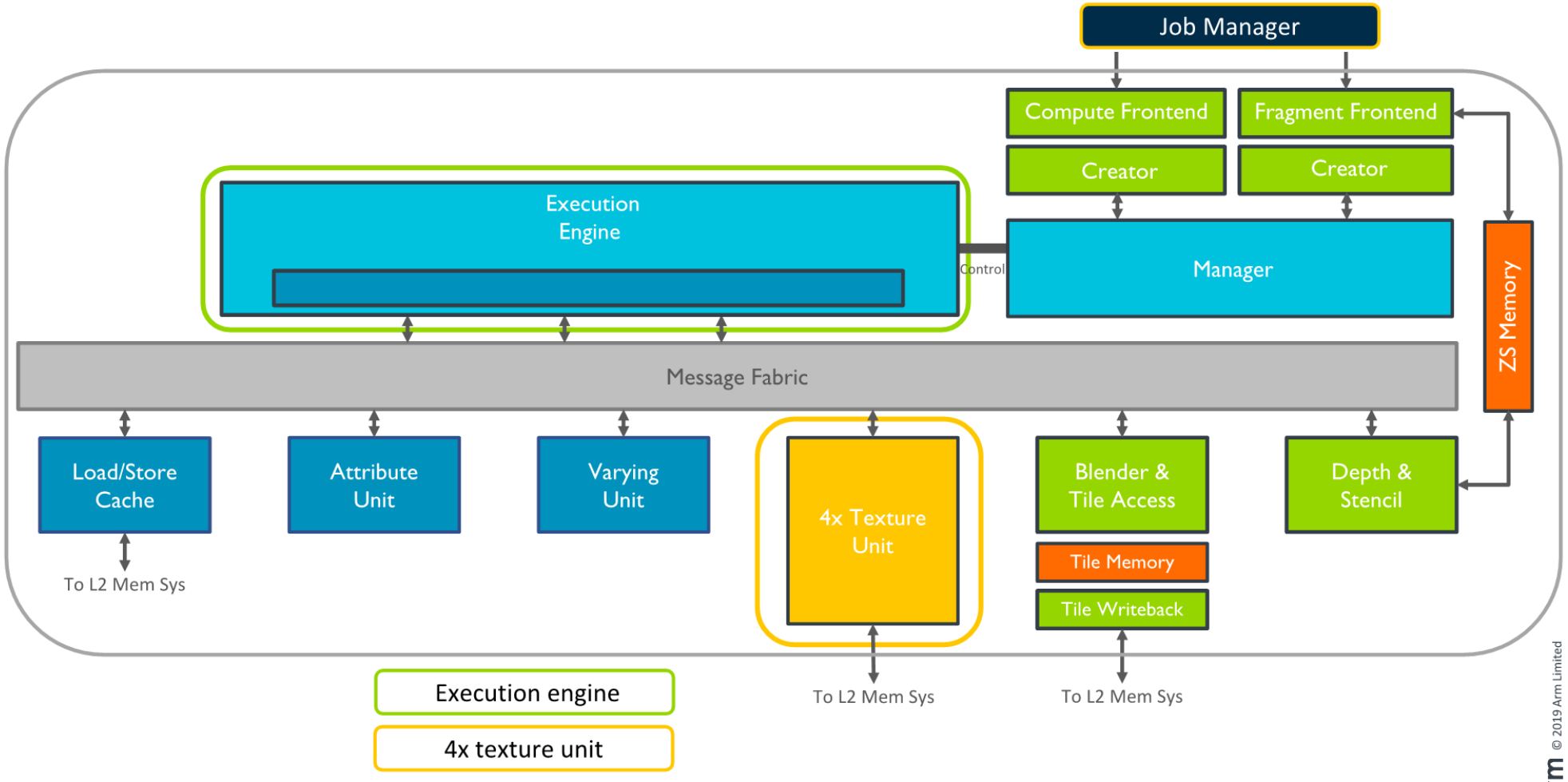
- The front-end manages warp state and instruction issue
- Issue capabilities and instruction latency vary by operation



The embargo for this content presented at Arm Tech Day will lift on Sunday, May 26 at 9:00 p.m. PT. Corresponding UK and Taiwan times are: Monday, May 27 at 5:00 a.m. BST / Monday, May 27 at 12:00 p.m. China Standard Time

arm © 2019 Arm Limited

Valhall Shader Core

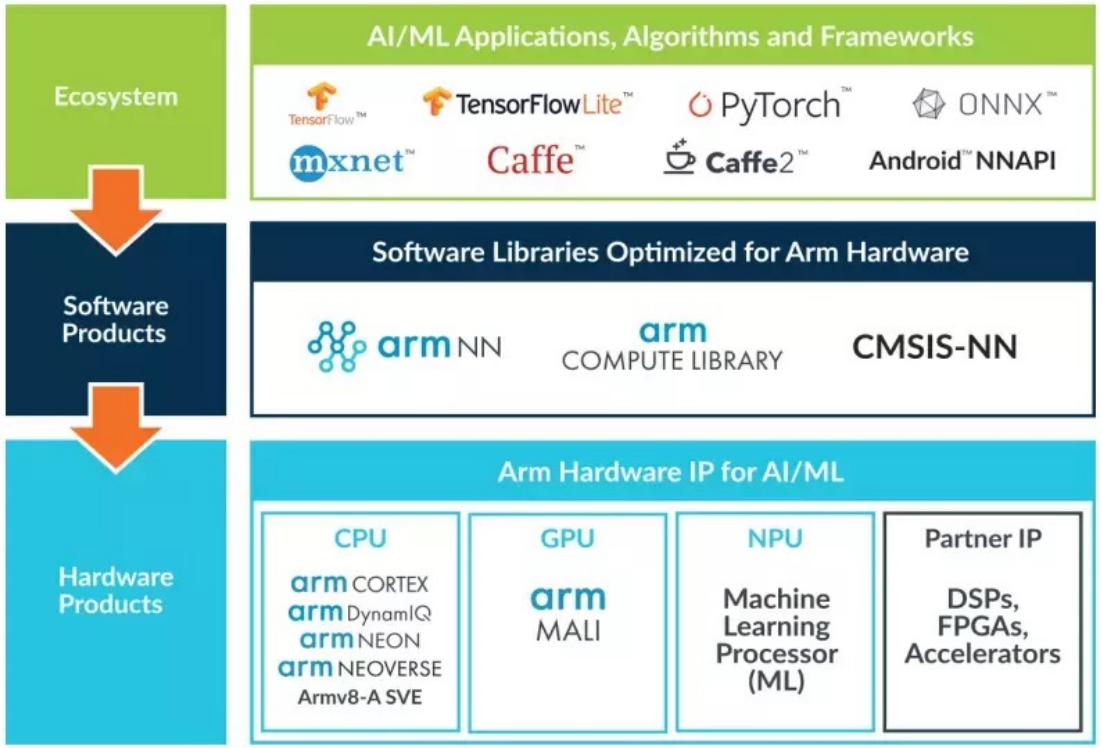


The embargo for this content presented at Arm Tech Day will lift on Sunday, May 26 at 9:00 p.m. PT. Corresponding UK and Taiwan times are: Monday, May 27 at 5:00 a.m. BST / Monday, May 27 at 12:00 p.m. China Standard Time

arm © 2019 Arm Limited

Software Stack for ARM GPUs

- Grphics APIs
  - OpenGL for 2D/3D graphics
  - Vulkan
- General-purpose programmiing models
  - OpenCL and Vulkan compute
  - OpenMP
- High-level libraries
  - ARM Compute Library (ACL)
    - Open-source, machine learning and computer vision library for ARM CPUs and Mali GPUs



https://community.arm.com/arm-community-blogs/b/architectures-and-processors-blog/posts/machine-learning-silicon-is-not-one-size-fits-all

Summary

- ARM GPU architectures evolving towards more modern and desktop GPU-like architectures
  - Scalar architecture
  - Shared control logic per execution engine
  - Hardware scheduler
  - Simplified compiler
  - Wider warps (4 → 8 → 16)
  - Full coherency for CPU and GPU

Architecture Trend

- Transistors are still getting smaller
  - More performance per unit area and power expected
- Integrate more domain-specific hardware accelerators
  - For performance: Tensor Cores, RT cores, memory-based accelerators, TMA
  - For programmability: cooperative groups, independent thread scheduling, thread block clusters
  - For security: VM support, hardware security support
- Scale out to larger clusters for DL training and HPC applications
  - Multi-GPU systems with NVLink and shared memory
  - Closer integration between CPUs, GPUs, and DPUs

- Challenges
  - Memory bandwidth and power constraints
  - Inter-node communication and synchronization overheads
  - Security and reliability concerns
  - Source and performance portability

Acknowledgments

- Slides are adapted from and inspired by
  - GPU computing teaching kit by NVIDIA
- Documents
  - NVIDIA, AMD, and ARM GPU reference manuals and white papers
  - https://www.amd.com/system/files/documents/rdna-whitepaper.pdf
  - https://community.arm.com/cfs-file/__key/telligent-evolution-components-attachments/01-2066-00-00-00-00-71-76/HotChips-Presentation-Jem-Davies.pdf
  - https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/white-papers/amd-cdna-3-white-paper.pdf
  - https://fileadmin.cs.lth.se/cs/Education/EDAN35/guestLectures/ARM-Mali.pdf
  - https://www.anandtech.com/show/14385/arm-announces-malig77-gpu
  - https://rocm.docs.amd.com/en/latest/
  - https://www.arm.com/products/development-tools/embedded-and-software/compute-library

Next Class

CUDA Basics